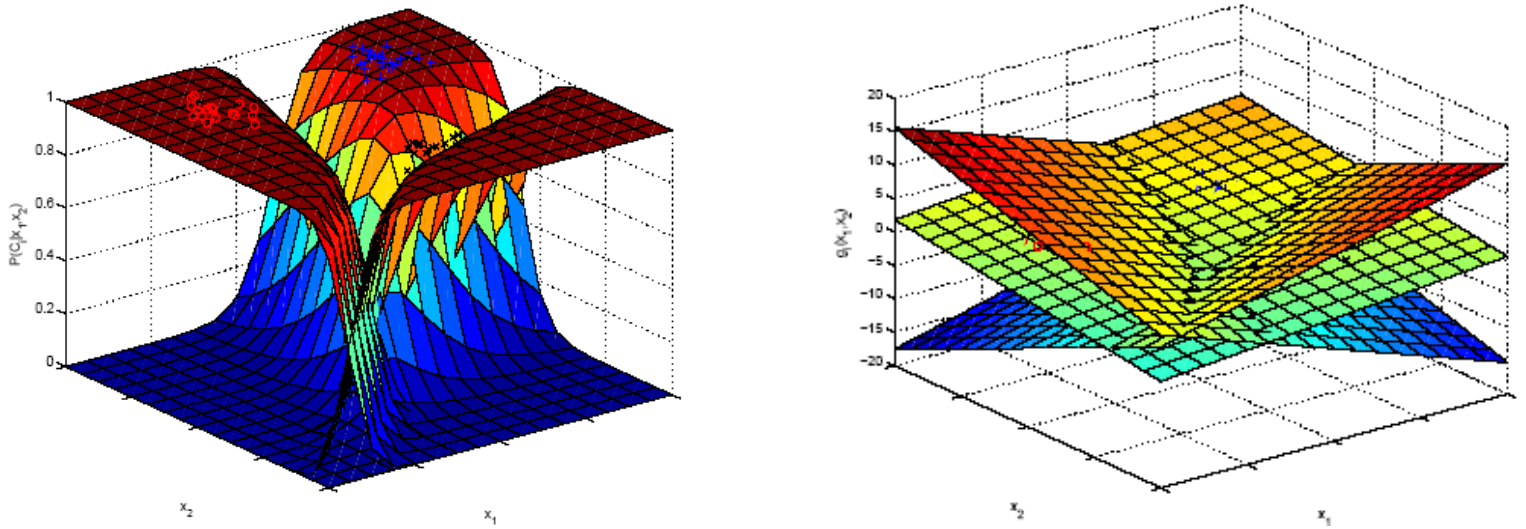
Machine Learning



Lecture 2: Classification & optimisation

Iasonas Kokkinos

Imperial College London

i.kokkinos@ic.ac.uk

Lecture 2: Classification & optimisation

Classification

Optimisation

- Gradient descent
- Newton–Raphson
- SGD
- L1 and L2 regularisation

Trees and ensembles

Decisions and evaluation

Binary cross-entropy: the training objective

The Bernoulli negative log-likelihood gives the per-example loss:

$$\ell(y_i, p_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

Add the losses for N independent training examples and fit w:

$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell(y_i, p_i), \qquad \hat{\mathbf{w}} = \arg\min_{\mathbf{w}} L(\mathbf{w})$$

The logistic gradient has a simple form

BCE loss: $$\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

Logistic regression: $$z_i = \mathbf{w}^T \mathbf{x}_i$$
$$p_i = g(\mathbf{w}^T \mathbf{x}_i)$$

Gradient with respect to weight vector:

$$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i, \qquad \nabla_{\mathbf{w}} \ell_i = (p_i - y_i)\mathbf{x}_i$$

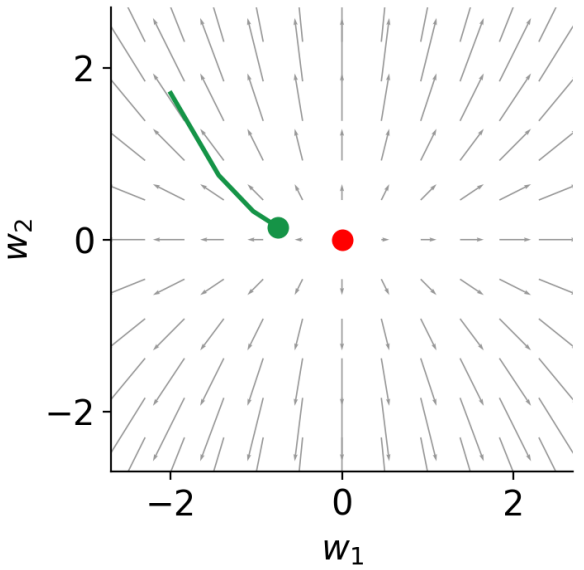
$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell_i, \qquad \nabla L(\mathbf{w}) = \sum_{i=1}^{N} (p_i - y_i)\mathbf{x}_i$$

$\nabla L(\widehat{\mathbf{w}}) = \mathbf{0}$ Nonlinear system of equations in w: Fit iteratively!

Numerical optimisation changes parameters

Classification supplies a loss L(w). An optimiser searches for parameter values that reduce it.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$$



Learning rate and narrow valleys

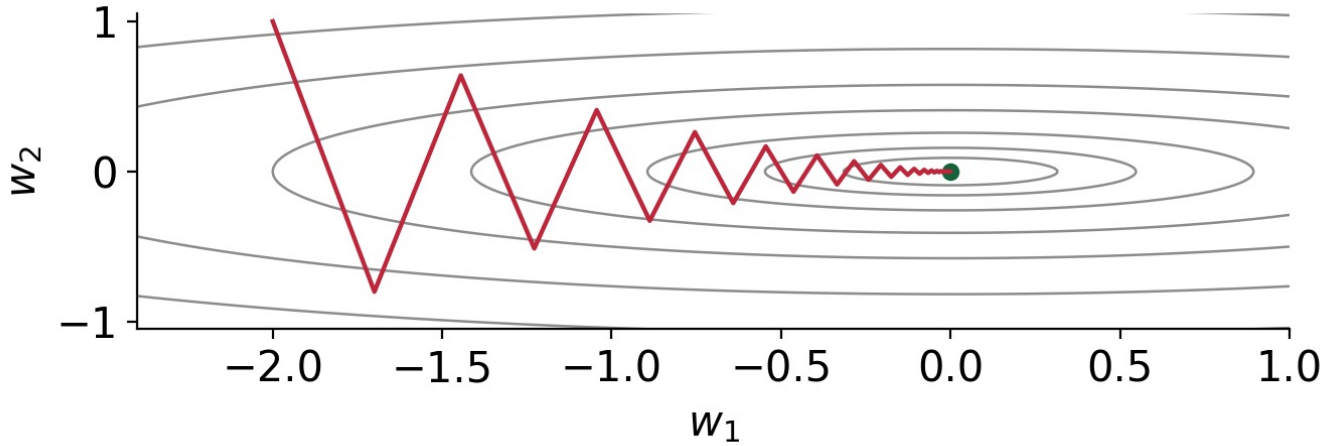
Step-size selection:

Initialize: $\mathbf{w}_0$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \nabla L(\mathbf{w}_t)$

How large a step?

Zig-zagging behavior:



A narrow valley has different curvature in different directions.
One rate can make progress slow or oscillatory.

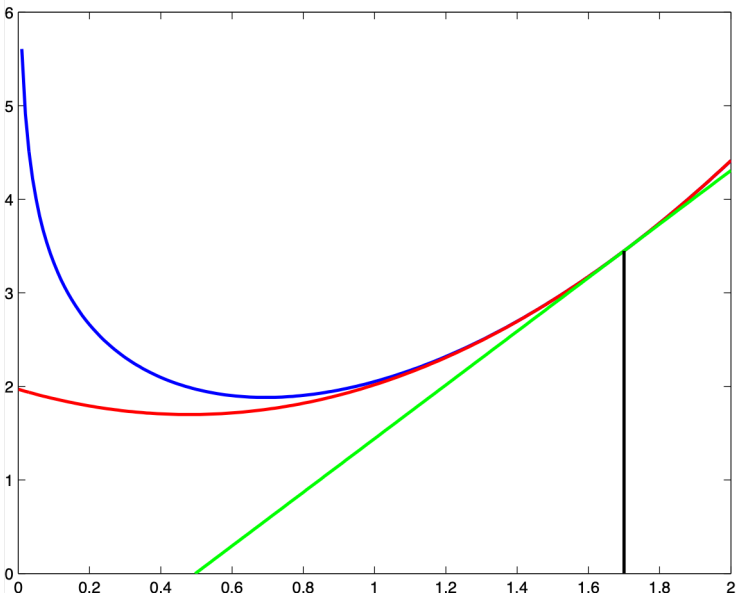
Linear and quadratic approximations

First-order Taylor series approximation:

$$L(w) \simeq L(a) + (w - a)L'(a)$$

Second-order Taylor series approximation:

$$L(w) \simeq L(a) + (w - a)L'(a) + \tfrac{1}{2}(w - a)^2 L''(a)$$



blue:

$$L(w) = w^2 - \log w + \exp(w/20)$$

green: linear approximation

$$l(w) = L(a) + (w - a)L'(a)$$

red: quadratic approximation

$$q(w) = l(w) + \tfrac{1}{2}(w - a)^2 L''(a)$$

a is the current parameter; l and q are local linear and quadratic models.

A quadratic model in several dimensions

First-order Taylor series approximation:

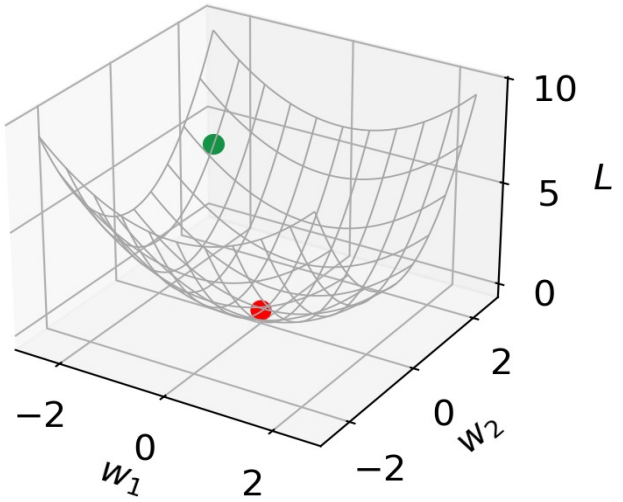
$$L(\mathbf{w}) \simeq L(\mathbf{w}_t) + (\mathbf{w} - \mathbf{w}_t)^T \nabla L(\mathbf{w}_t)$$

Second-order Taylor series approximation:

$$q_t(\mathbf{w}) = L(\mathbf{w}_t) + \mathbf{d}^T \nabla L(\mathbf{w}_t) + \frac{1}{2}\mathbf{d}^T \mathbf{H}_t \mathbf{d}$$

$$\mathbf{d} = \mathbf{w} - \mathbf{w}_t$$

$$(H_t)_{jk} = \left. \frac{\partial^2 L}{\partial w_j \partial w_k} \right|_{\mathbf{w}_t}$$



H_t contains second derivatives: it describes local curvature.
With D+1 coefficients, it has D+1 rows and columns; j and k index these.

Newton's method in several dimensions

Initial parameter: $\mathbf{w}_0$

At any point, form quadratic approximation:

$$q_t(\mathbf{w}) = L(\mathbf{w}_t) + \mathbf{d}^T \nabla L(\mathbf{w}_t) + \tfrac{1}{2}\mathbf{d}^T \mathbf{H}_t \mathbf{d}$$

Stationary condition for the quadratic model:

$$\nabla q_t(\mathbf{w}) = \mathbf{0} \quad \Longrightarrow \quad \mathbf{H}_t \mathbf{d} = -\nabla L(\mathbf{w}_t)$$

Take the stationary point of this quadratic model.

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{H}_t^{-1} \nabla L(\mathbf{w}_t)$$

Check convergence and whether the loss decreases.

Newton–Raphson for logistic regression

Gradient: $$\frac{\partial L}{\partial w_k} = \sum_{i=1}^{N} (p_i - y_i) x_{ik}$$

Hessian:

$$\frac{\partial^2 L}{\partial w_k \partial w_j} = \frac{\partial}{\partial w_j} \left[\sum_{i=1}^{N} (p_i - y_i) x_{ik} \right]$$

$$= \sum_{i=1}^{N} x_{ik} \frac{\partial p_i}{\partial w_j} \qquad = \sum_{i=1}^{N} x_{ik}\, p_i (1 - p_i) x_{ij}$$

$p_i = g(w^T x_i)$. i indexes examples; j and k index coefficients.

Summation- and matrix-based expressions

$$H_{kj} = \frac{\partial^2 L}{\partial w_k \partial w_j} = \sum_{i=1}^{N} x_{ik} p_i (1 - p_i) x_{ij}$$

Matrix version of same result:

$$\mathbf{H}(\mathbf{w}) = \mathbf{X}^T \mathbf{R} \mathbf{X}, \qquad R_{ii} = p_i (1 - p_i)$$

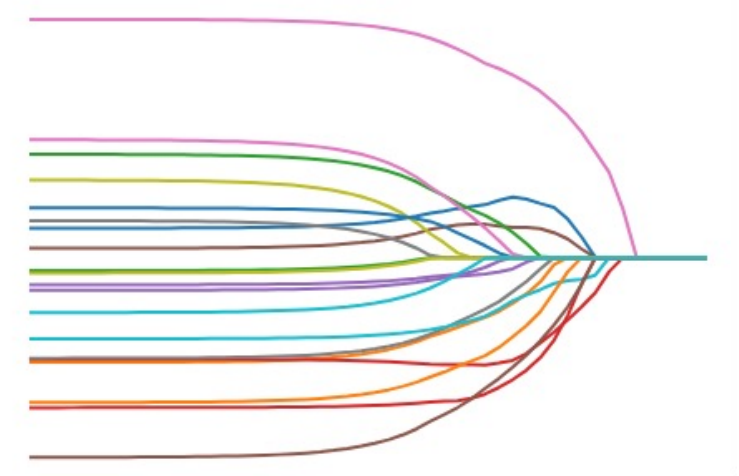
R is an N × N diagonal matrix, with $R_{ii} = p_i(1-p_i)$.
X is N × (D+1); p_i is the predicted probability of label 1.

Lecture 2: Classification & optimisation

Classification

Optimisation

- Gradient descent
- Newton–Raphson
- SGD
- **L1 and L2 regularisation**



Trees and ensembles

Decisions and evaluation

Murphy, Probabilistic Machine Learning: An Introduction
§§11.4.1–11.4.2: L1 regularisation and sparsity

LASSO: an L1 coefficient penalty

LASSO (Least Absolute Shrinkage and **Selection** Operator) selects a subset of the features: some coefficients can become exactly zero.

Fit the data while penalising the L1 norm of the parameter vector

$$J_{\text{lasso}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}_{1:D}\|_1$$

L1 norm

$$\|\mathbf{w}_{1:D}\|_1 = \sum_{j=1}^{D} |w_j|$$

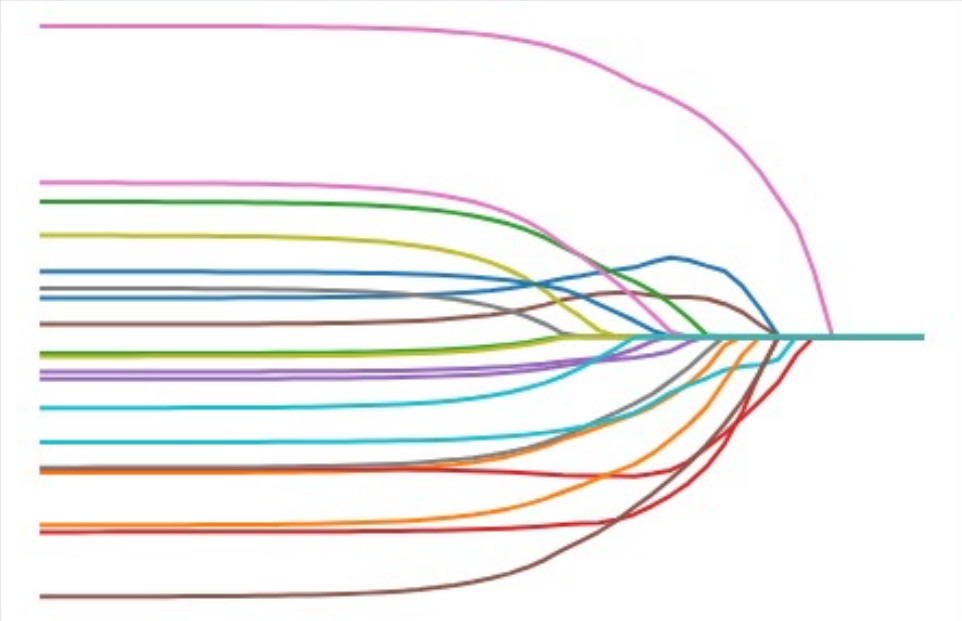
$w_{1:D}$ excludes the intercept w_0.
The intercept is not penalised.

Tibshirani (1996), JRSS B

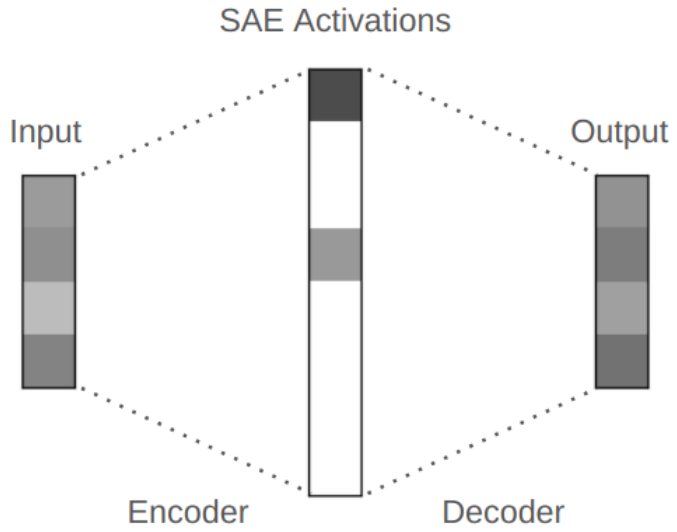
L1 sparsity and interpretability

Selecting features for prediction

Zero coefficients remove features from the fitted predictor.
Selection concerns predictive value given the other features.



LASSO with increasing λ



Cunningham et al. (2023): sparse autoencoders (Lecture 6)

https://arxiv.org/abs/2309.08600

L2 and L1: how coefficients shrink

L2: squared penalty

$$\Omega_2(\mathbf{w}) = \frac{\lambda}{2} \sum_{j=1}^{D} w_j^2,$$

$$\frac{\partial \Omega_2}{\partial w_j} = \lambda w_j$$

The gradient gets smaller as the coefficient gets smaller. Shrinkage slows near zero.

Gradient step (penalty alone)

$$w_{t+1,j} = (1 - \eta\lambda) w_{t,j}$$

For $0 < \eta\lambda < 1$, weights decay. Coefficients usually stay nonzero.

L1: absolute-value penalty

$$\Omega_1(\mathbf{w}) = \lambda \sum_{j=1}^{D} |w_j|$$

$$\frac{\partial \Omega_1}{\partial w_j} = \lambda \ \mathrm{sign}(w_j)$$

For a nonzero coefficient, the gradient magnitude is constant. The pull stays strong near zero.

Thresholding step (penalty alone)

$$|w_{t+1,j}| = \max(|w_{t,j}| - \eta\lambda, 0)$$

Subtract $\eta\lambda$ from the magnitude, clip at zero: some features drop out.

Regularisation penalties and constraints

LASSO penalises L1. Ridge penalises L2 squared.

Penalty formulation

$$\min_{\mathbf{w}} \ \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}_{1:D}\|_1$$

Constraint formulation

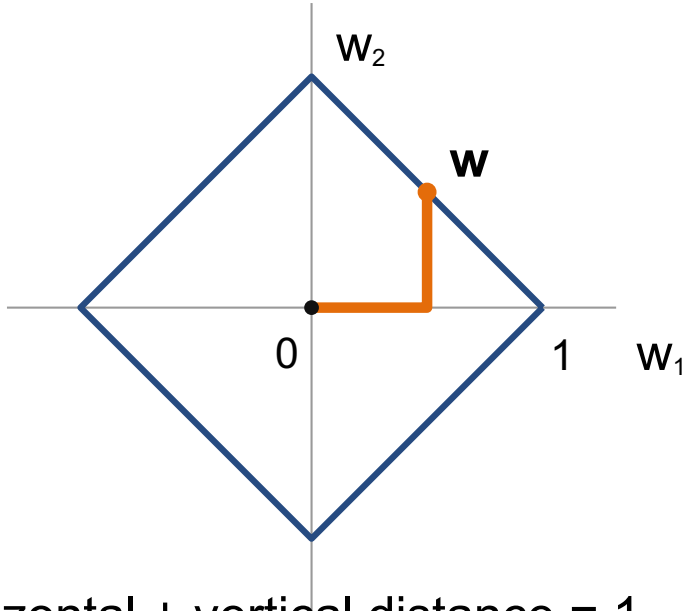
$$\min_{\mathbf{w}} \ \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \quad \text{s.t.} \quad \|\mathbf{w}_{1:D}\|_1 \leq r$$

r ≥ 0 is the constraint radius. s.t. means subject to.
For each penalty strength, compare its corresponding constraint radius.

Unit balls: L1 and L2 norms

L1: Manhattan distance

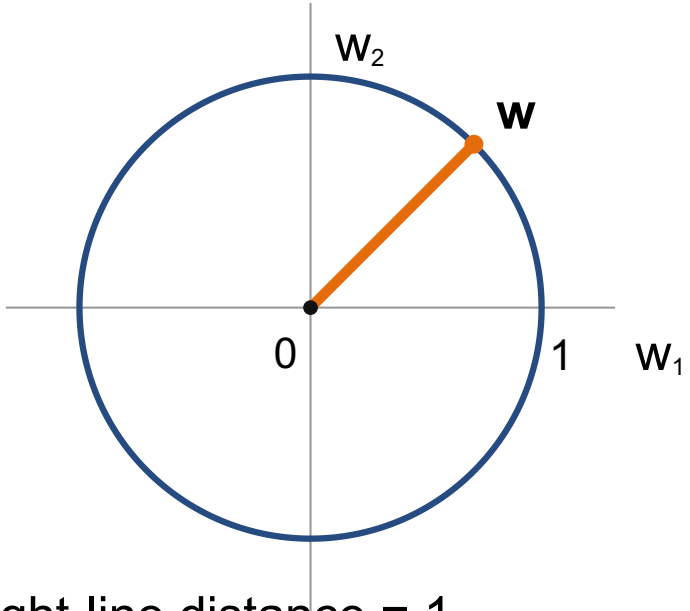
$$\|\mathbf{w}_{1:D}\|_1 = \sum_{j=1}^{D} |w_j|$$



Horizontal + vertical distance = 1

L2: Euclidean distance

$$\|\mathbf{w}_{1:D}\|_2 = \left(\sum_{j=1}^{D} w_j^2 \right)^{1/2}$$

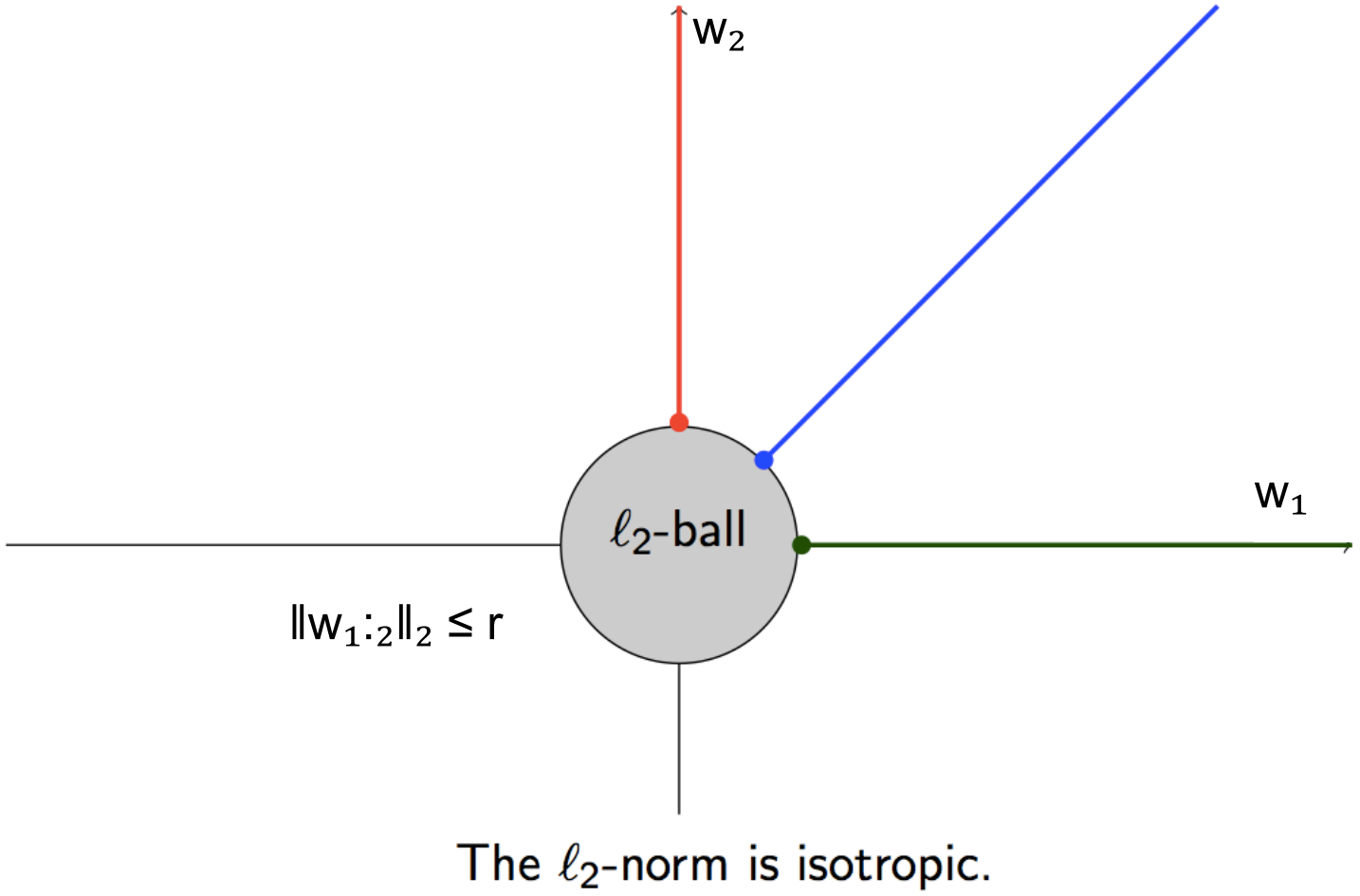


Straight-line distance = 1

Boundary: norm = 1. Unit ball: norm ≤ 1.

L2 geometry: projection onto a ball

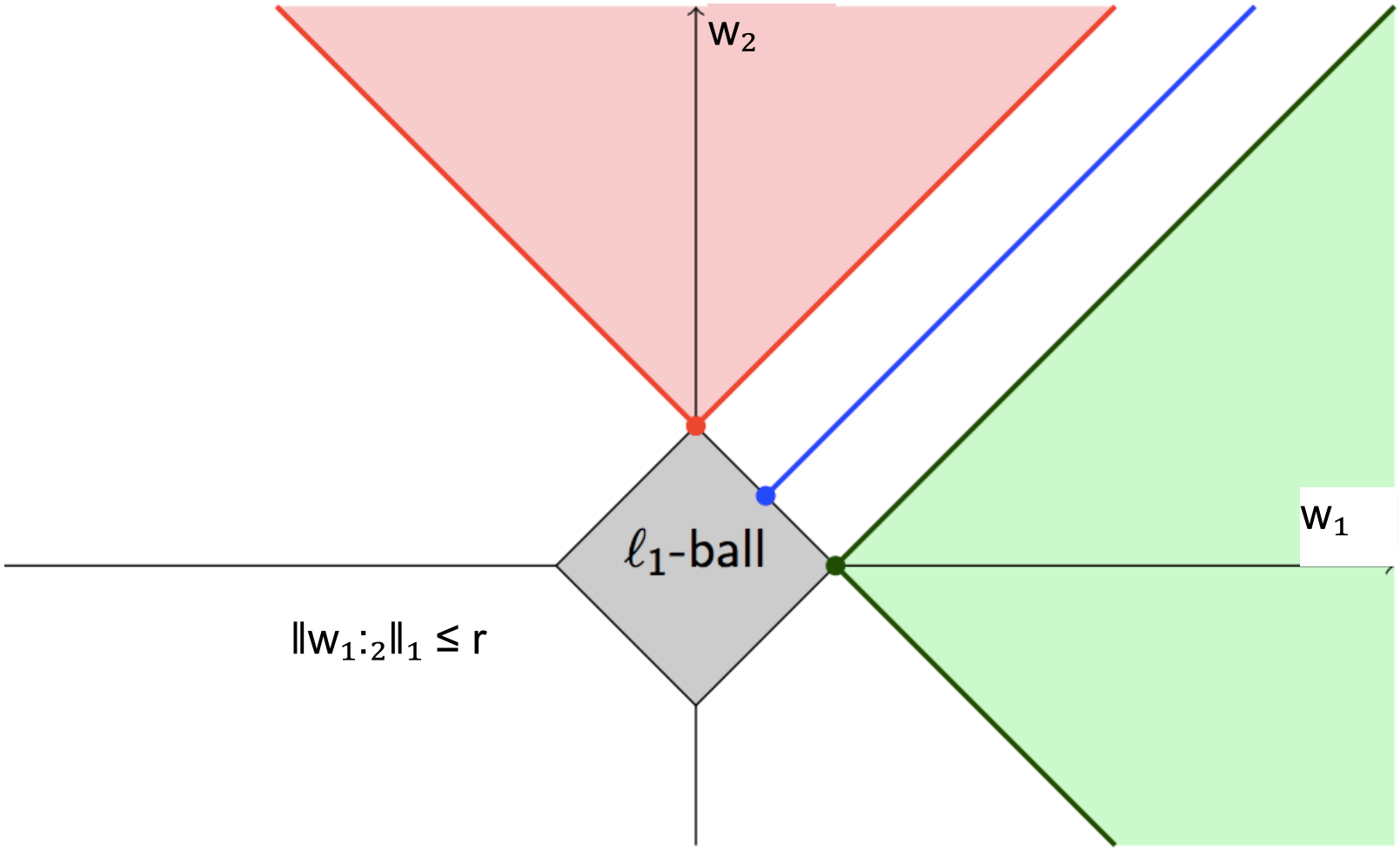
Euclidean projection illustration



The nearest point usually has several nonzero coordinates.

L1 geometry: projection onto corners

Euclidean projection illustration



Many coefficient vectors project onto a corner, where one coordinate is zero.

What we have learned so far

Loss functions

Squared L2 loss for regression, cross-entropy for classification

Regularisation

L2 shrinks coefficients. L1 encourages sparsity. Probabilistic interpretation.

Minimizers

Closed form, second-order methods, gradient descent and SGD

ML fundamentals: rooted in probability, statistics and optimisation.

Hastie, Tibshirani & Friedman (2009), The Elements of Statistical Learning.

Statistical foundations: historical context

Bayesian parameter estimation **1763–1774**

Bayes (1763), Laplace (1774): infer unknown parameters from data.

Linear regression **1805–1809**

Legendre (1805), Gauss (1809): least squares.

Logistic regression **1944–1958**

Berkson (1944), Cox (1958): model binary-outcome probabilities.

Ridge regression & Gaussian priors **1962–1970**

Hoerl (1962), Hoerl & Kennard (1970): coefficient shrinkage.

LASSO **1996**

Tibshirani (1996): L1 shrinkage and feature selection.

Lecture 2: Classification & optimisation

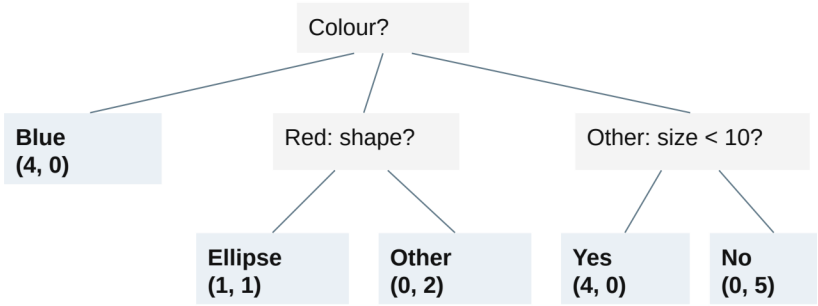
Classification

Optimisation

Trees and ensembles

- **Decision trees**
- Ensembles and bagging
- Random forests

Decisions and evaluation

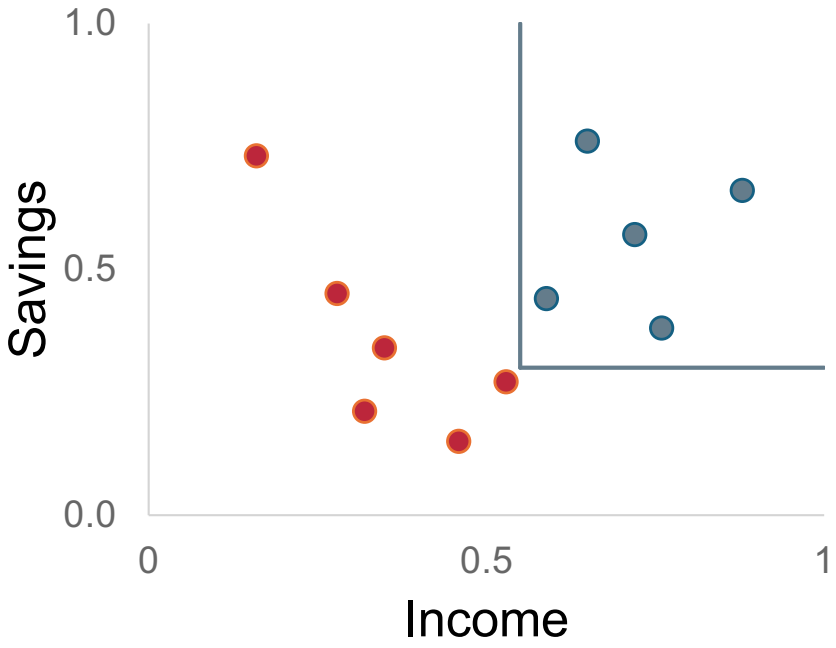


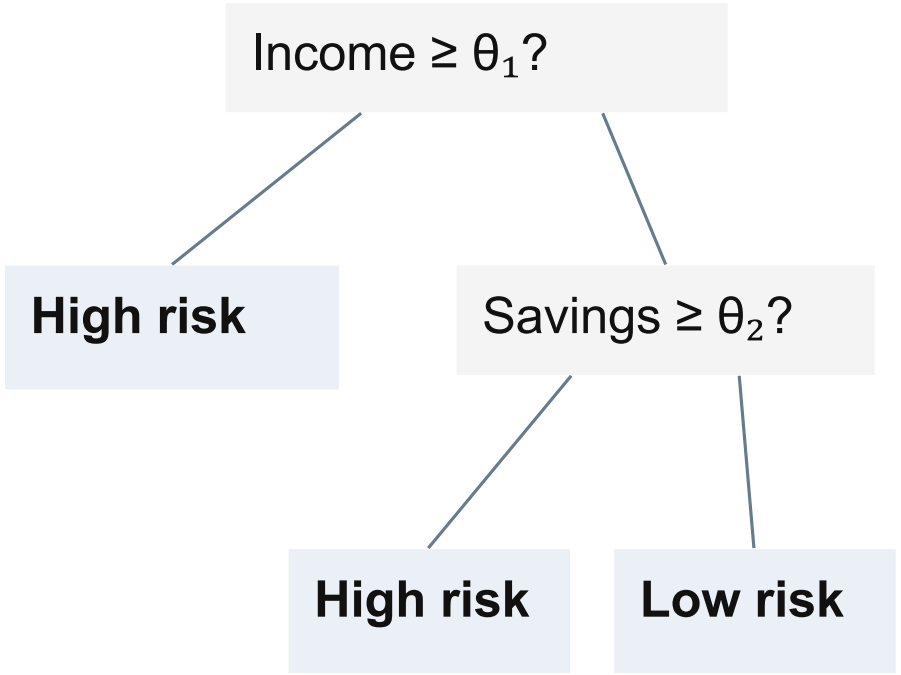
Murphy, Probabilistic Machine Learning: An Introduction
§18.1: classification and regression trees

Decision tree:

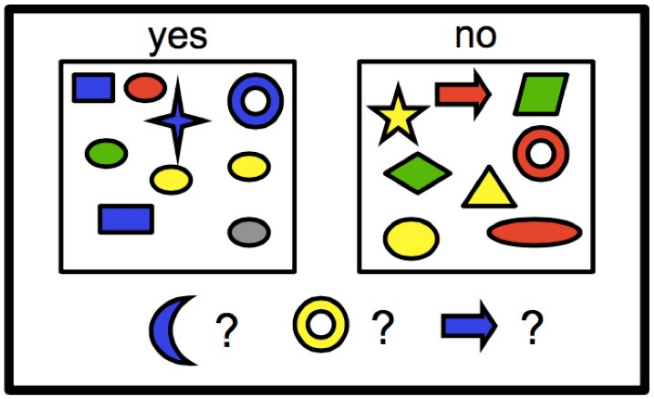
Income and savings define a simple risk classification example.

Red: high risk. Blue: low risk.



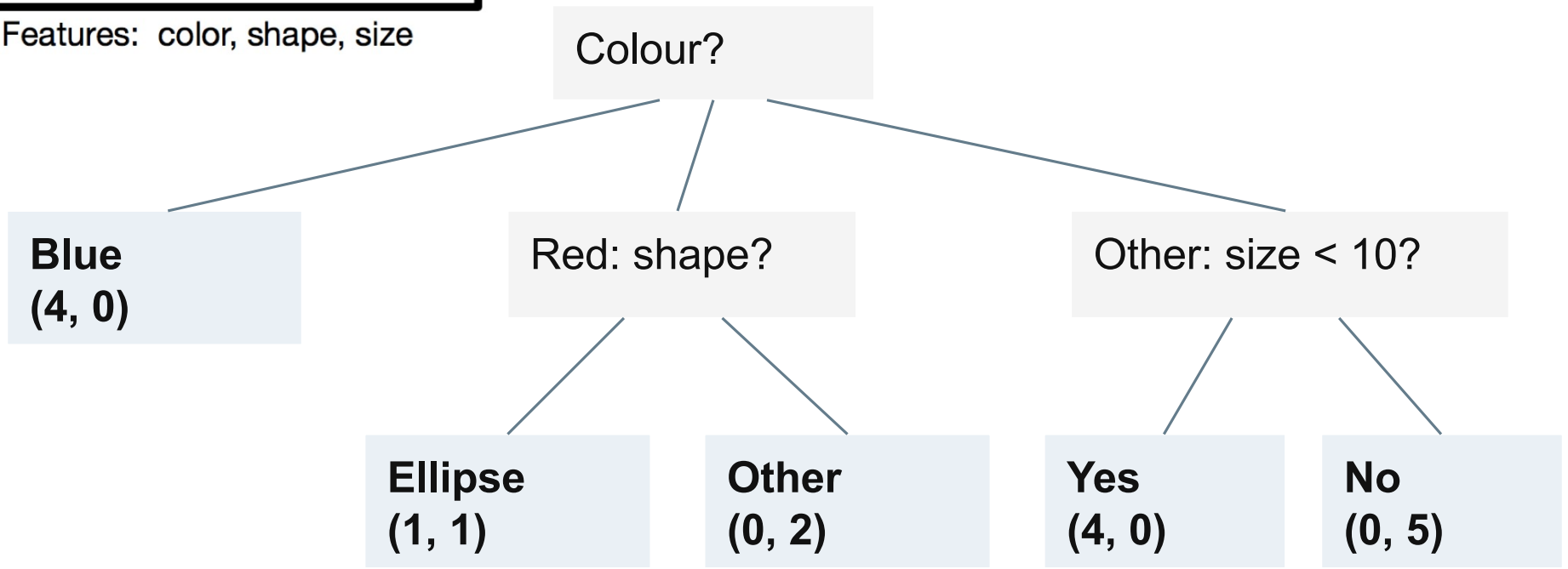


A tree predicts from its leaves



Features: color, shape, size

Inputs have colour, shape and size.
A test sends each input to a child.



Each pair is (yes-label count, no-label count) at a leaf.
Predict by the leaf's class frequencies, or choose the majority label.
Depth counts tests along a path

Decision-tree models and learning

A leaf is a region R_j. Fit its distribution using the training cases in that leaf.

Regression $$p(y_i \mid \mathbf{x}_i \in \mathcal{R}_j) = \mathcal{N}(y_i \mid \mu_j, \sigma_j^2)$$

Binary classification $$p(y_i \mid \mathbf{x}_i \in \mathcal{R}_j) = \text{Bernoulli}(y_i \mid \mu_j)$$

μ_j is the leaf mean or class-1 probability. σ_j^2 is the regression variance.

$$J(T) = -\sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i; T) + \lambda C(T)$$

C(T) measures tree complexity, such as the number of nodes. $\lambda \geq 0$.

Greedy learning

1. Start with a leaf and search candidate feature tests and thresholds.
2. Keep the best local split, then repeat at each child.
3. Stop or prune to control complexity. Choose settings by validation.

Q: how to split?

Entropy of a distribution

Information measures surprise.

$$I(c) = \log_2\left(\frac{1}{p_c}\right) = -\log_2 p_c$$

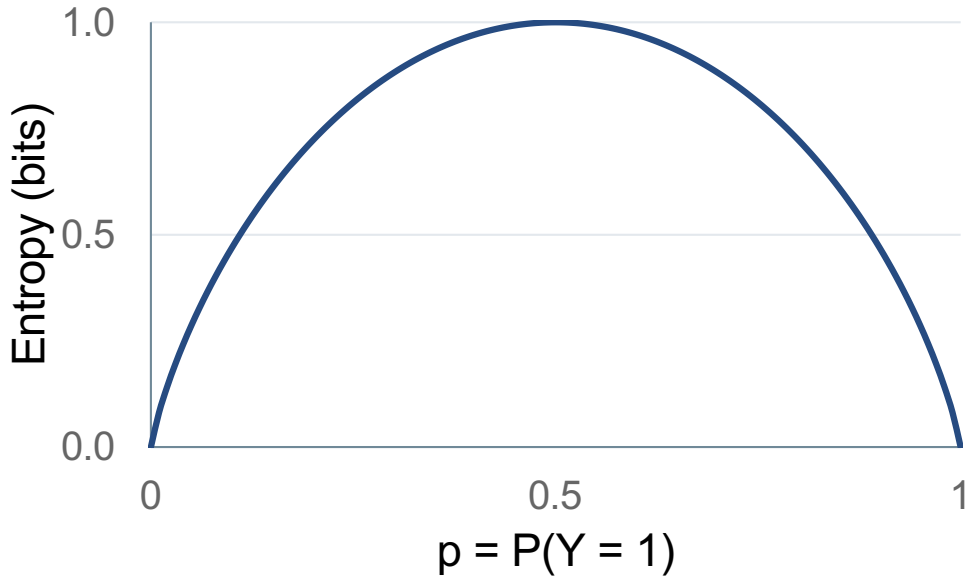
Entropy is its expected value.

$$H = -\sum_{c=1}^{C} p_c \log_2 p_c = \mathbb{E}[I(c)]$$

For binary labels, p is the probability of class 1.

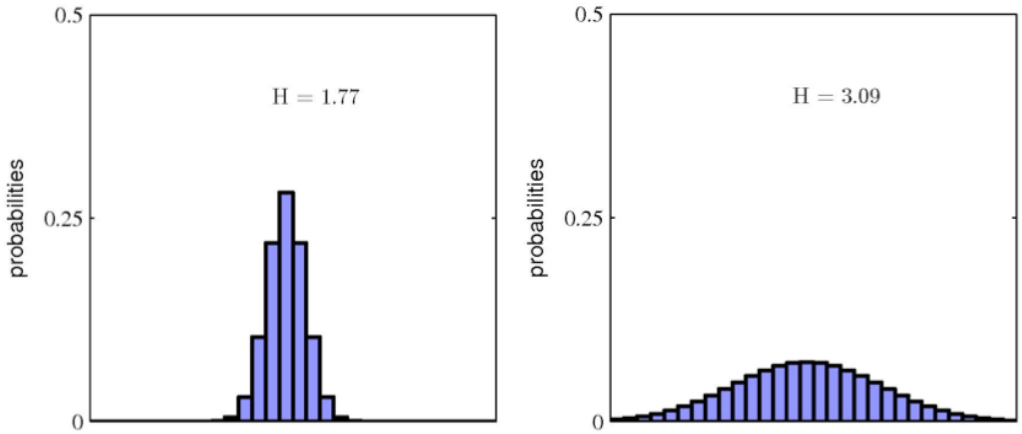
A pure node has no class uncertainty.

An even split is maximally uncertain.



H = 0 at p = 0 or 1. H = 1 bit at p = 0.5. Use 0 $\log_2$ 0 = 0.

Entropy measures uncertainty



- Histograms of two probability distributions over 30 bins.
- The largest entropy will arise from a uniform distribution
  H = -ln(1/30) = 3.40.

For class labels Y: p_c = P(Y = c). There are C classes. Uniform: H = log C.

Information gain at the current node

Count the class labels in this local set S to estimate the entropy of the node distribution

$$H(S) = -\sum_{c=1}^{C} p_c \log_2 p_c, \qquad p_c = \frac{n_c}{|S|}$$

Split S by the candidate attribute, then weight each left/right child distribution entropy by its size.

$$\mathrm{IG} = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R)$$

Parent entropy minus weighted child entropy

Choose the largest gain among candidates at the same parent.

The fruit at the root

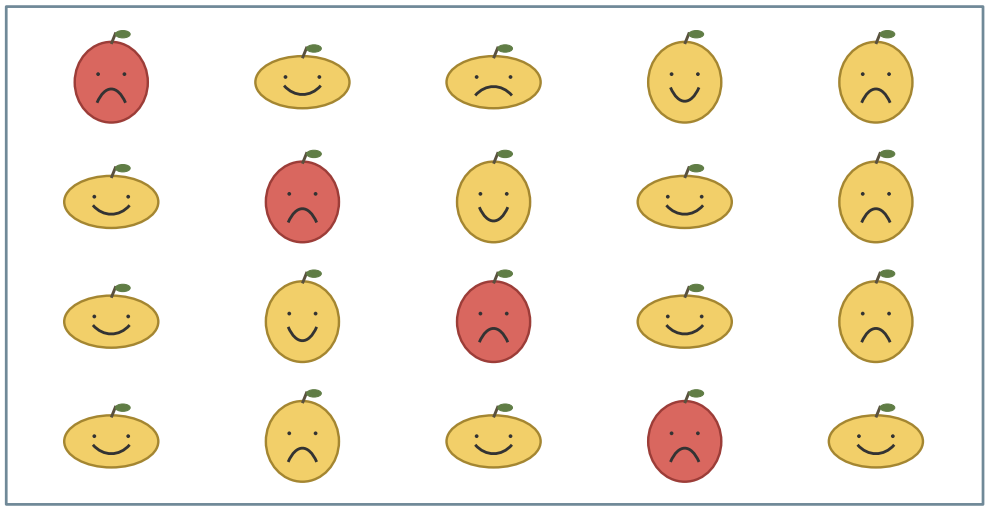
Before any split, S contains all the training examples.

Colour (input) Red Yellow

Firmness (input) Soft Firm

Taste (target) Sweet Sour

S: all 20 fruit, before splitting



11 sweet, 9 sour

Class probabilities are their fractions within S.

$$p_{\text{sweet}} = \frac{11}{20}, \qquad p_{\text{sour}} = \frac{9}{20}$$

Entropy measures the uncertainty of these class labels.

$$H(S) = -\frac{11}{20}\log_2\frac{11}{20} - \frac{9}{20}\log_2\frac{9}{20} = 0.993 \text{ bits}$$

Comparing two candidate splits

Predict taste from colour and firmness. Each fruit below shows all three.

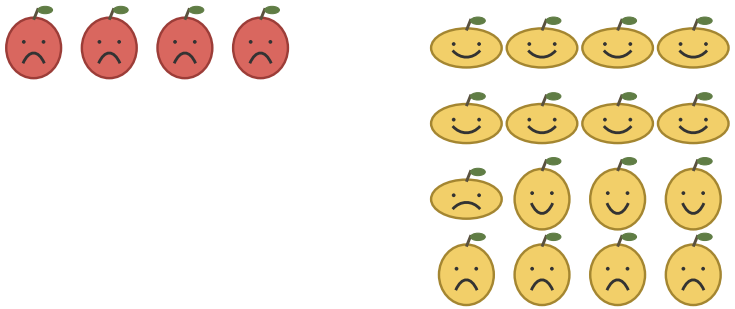
Colour (input) Red, Yellow

Firmness (input) Soft, Firm

Taste (target) Sweet, Sour

Same 20 fruit: 11 sweet, 9 sour. Parent entropy = 0.993 bits.

A: split by colour

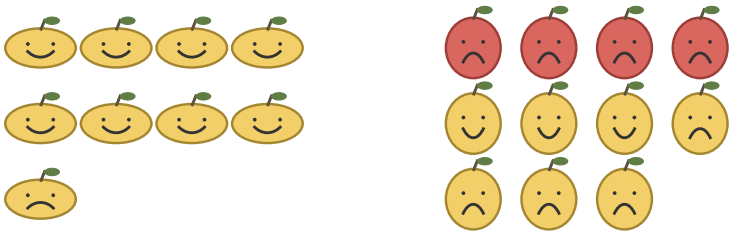


0 sweet, 4 sour
H = 0

11 sweet, 5 sour
H = 0.896

Weighted entropy = 0.717 bits
Information gain = 0.276 bits

B: split by firmness



8 sweet, 1 sour
H = 0.503

3 sweet, 8 sour
H = 0.845

Weighted entropy = 0.691 bits
Information gain = 0.301 bits

Choose B: a pure red group alone does not make A the better split.

A split below the root

Predict whether fruit is sweet or sour. The root splits by firmness.
Now consider only the 11 firm fruit: 3 sweet and 8 sour.

Current branch	Sweet	Sour	Total
Red	0	4	4
Other colour	3	4	7
Parent: firm	3	8	11

For the firm-fruit node: H = 0.8454 bits. Split it by colour:

$$\text{Gain} = 0.8454 - \frac{4}{11}(0) - \frac{7}{11}(0.9852) = 0.2184 \text{ bits}$$

Compute the gain using these 11 fruit. The 9 soft fruit follow the other branch and do not enter this calculation.

Pruning a decision tree

Grow the tree, then remove splits that do not help on validation data.

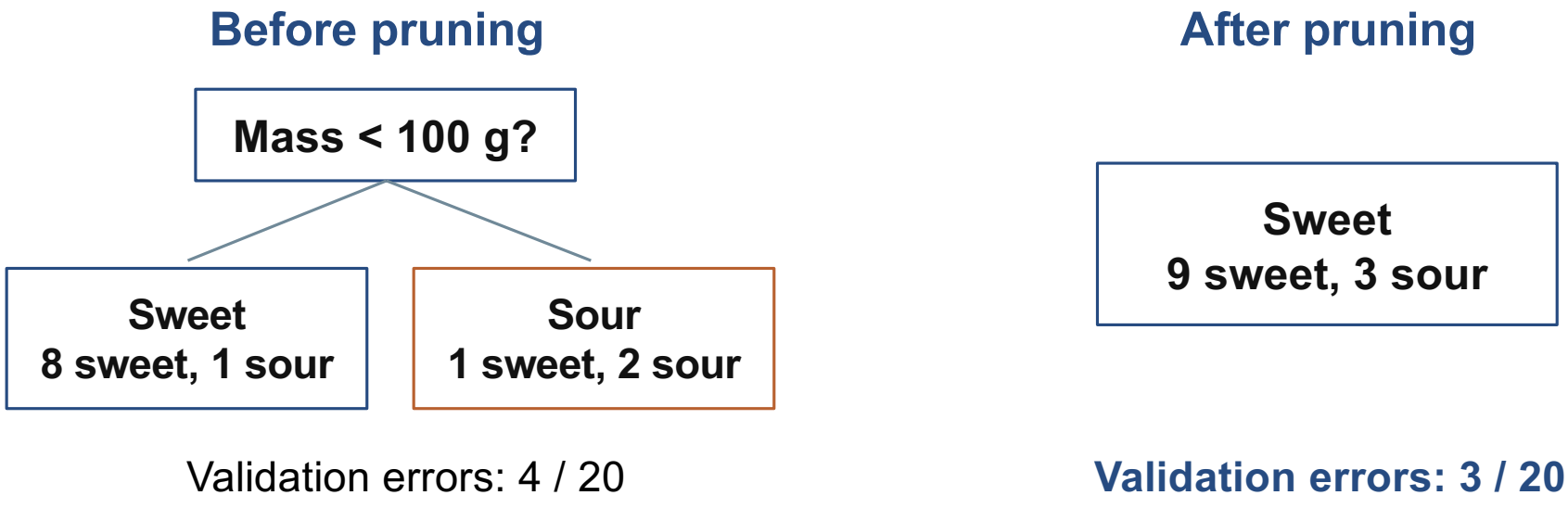


Illustration: leaf counts use training data; errors use 20 validation cases.

1. Visit a node whose children are leaves.
2. Replace its split with a leaf fitted on its training cases.
3. Keep the change if validation performance improves.
4. Work upwards and repeat.

For classification, the replacement leaf predicts the training majority class.

Training error and validation error

Tree construction	Training	Validation	Diagnosis
Early stopping	0.2975	0.4588	Underfit
Exhaustive growth	0.0142	0.3763	Overfit
Pruning	0.1025	0.1997	Best of these

Limit depth or increase minimum leaf size to control complexity.
Pre-pruning stops growth. Post-pruning simplifies a grown tree.

Compare validation error as well as the train–validation gap.

Decision trees: strengths and limits

What makes trees useful

Greedy training is fast. Prediction follows a short path of tests.
Tests can use different feature types, such as colour, size or shape.

How we control complexity

Choose depth, minimum leaf size and pruning using validation data.
Compare held-out performance with the training error.

Why consider ensembles next?

A deep tree can overfit and change sharply with its training sample.
Combining several trees can reduce this instability.

Lecture 2: Classification & optimisation

Classification

Optimisation

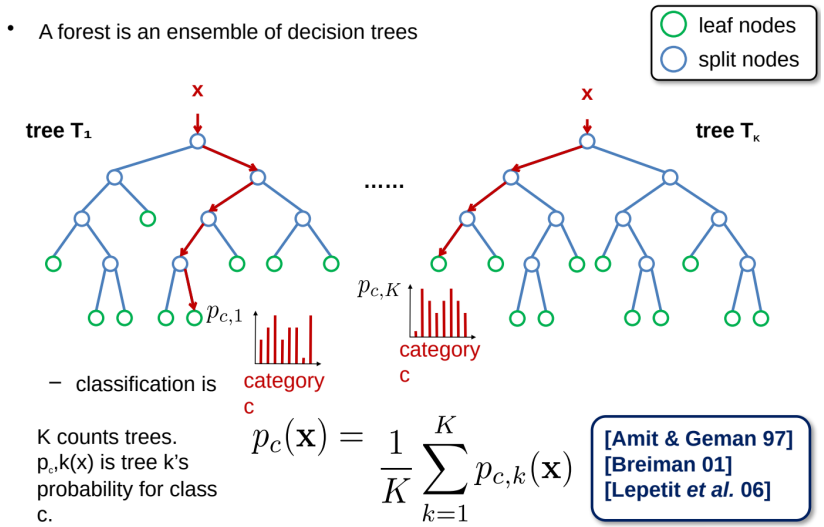
Trees and ensembles

Decision trees

Ensembles and bagging

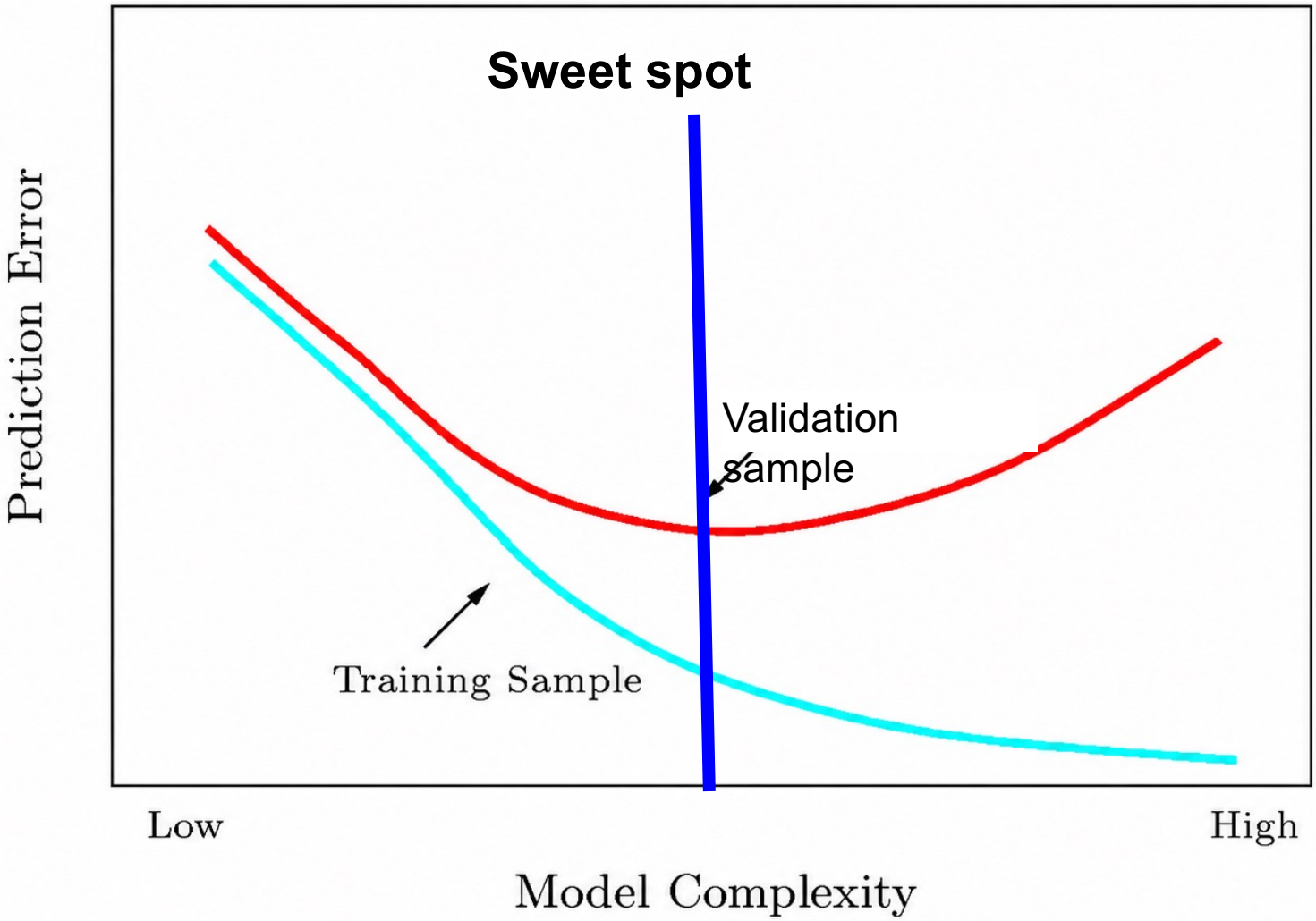
Random forests

Decisions and evaluation



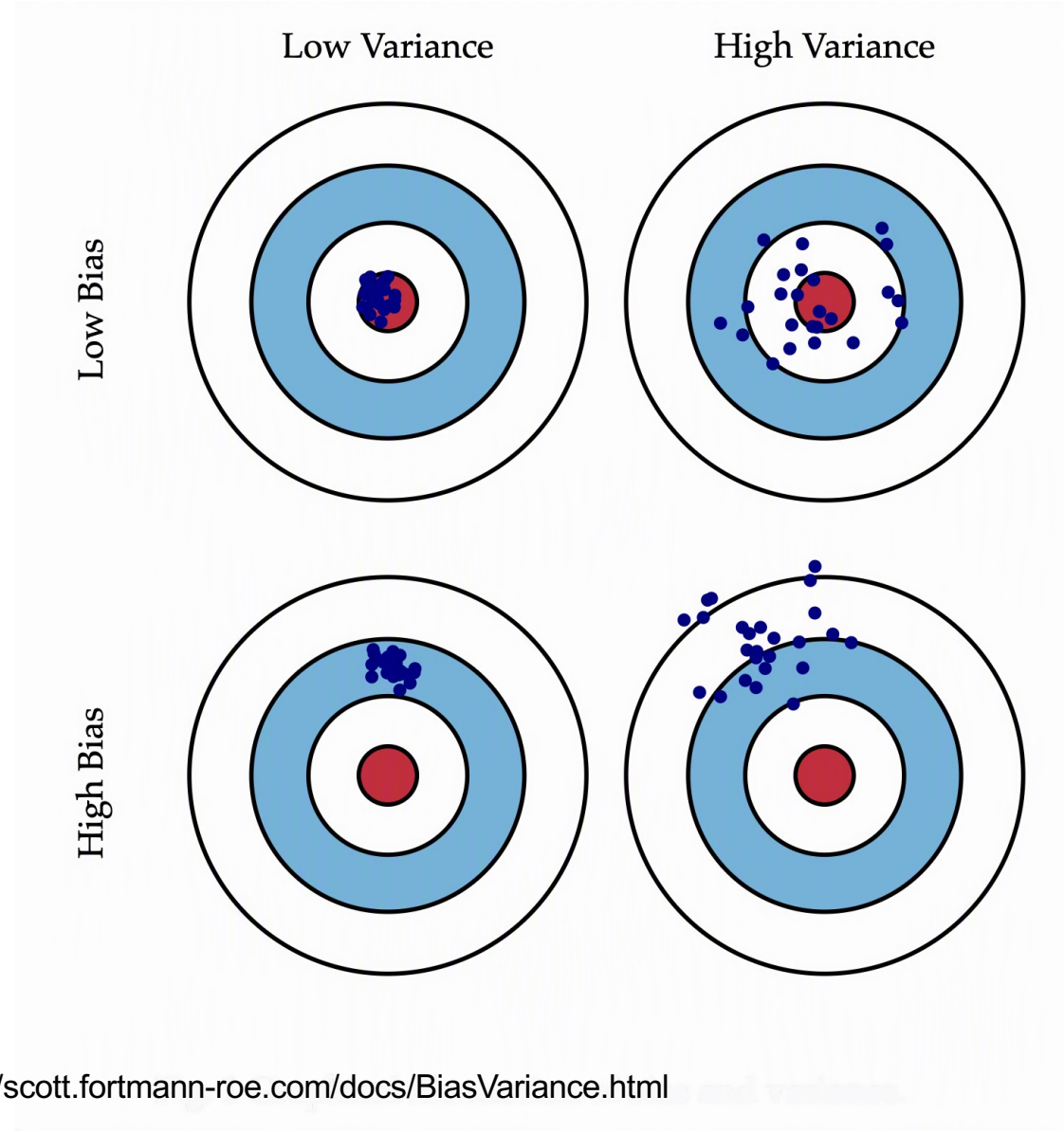
Murphy, Probabilistic Machine Learning: An Introduction
§4.7.6: bias–variance; §18.3: bagging

Model complexity and held-out error



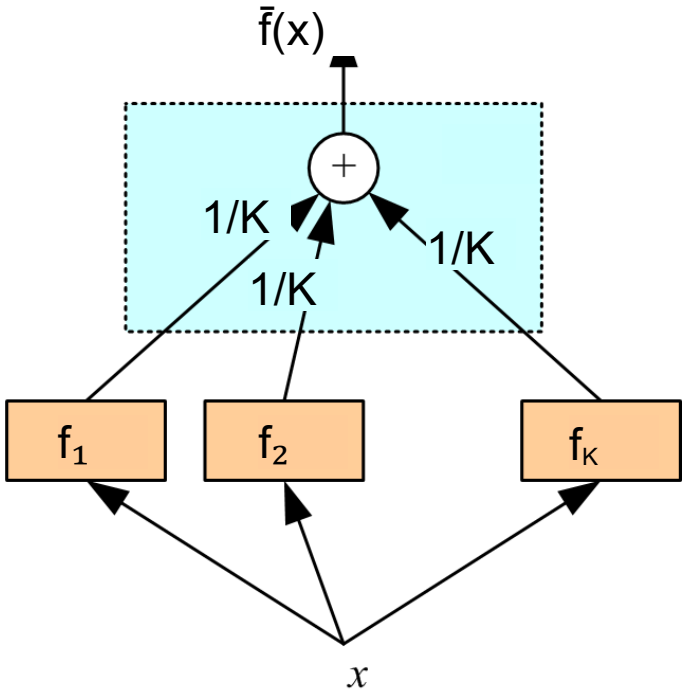
Larger ridge λ means lower complexity. The held-out curve is schematic.

Bias and variance - intuition



https://scott.fortmann-roe.com/docs/BiasVariance.html

A committee can reduce variance



At input x, average K learners.
$f_k(x)$ is learner k's prediction.

$$\bar{f}(x) = \frac{1}{K}\sum_{k=1}^{K} f_k(x)$$

Averaging models with the same expected prediction leaves their bias unchanged.

For uncorrelated fluctuations with equal variance:

$$\mathrm{Var}[\bar{f}(x)] = \frac{\mathrm{Var}[f_k(x)]}{K}$$

Positive correlation reduces the gain. We need diverse learners.

Error of an average predictor

K predictors $f_k(x)$ form a committee.
$f^*(x)$ is the true function and $\varepsilon_k(x)$ the error of predictor k.

$$\overline{f}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} f_k(\mathbf{x}), \qquad f_k(\mathbf{x}) = f^*(\mathbf{x}) + \epsilon_k(\mathbf{x})$$

$$E_k = \mathbb{E}_{\mathbf{x}}\left[\epsilon_k(\mathbf{x})^2\right], \qquad E_{\mathrm{AV}} = \frac{1}{K}\sum_{k=1}^{K} E_k$$

E_k is expected squared error. E_{AV} averages it over the K predictors.

$$E_{\mathrm{COM}} = \mathbb{E}_{\mathbf{x}}\left[(\overline{f}(\mathbf{x}) - f^*(\mathbf{x}))^2\right] = \mathbb{E}_{\mathbf{x}}\left[\left(\frac{1}{K}\sum_{k=1}^{K}\epsilon_k(\mathbf{x})\right)^2\right]$$

Uncorrelated errors can average out

Assume zero-mean errors, uncorrelated across different predictors.

$$\mathbb{E}_{\mathbf{x}}[\epsilon_k(\mathbf{x})\epsilon_{k'}(\mathbf{x})] = 0 \quad \text{for } k \neq k'$$

Expand the square. Cross terms vanish for k ≠ k′.

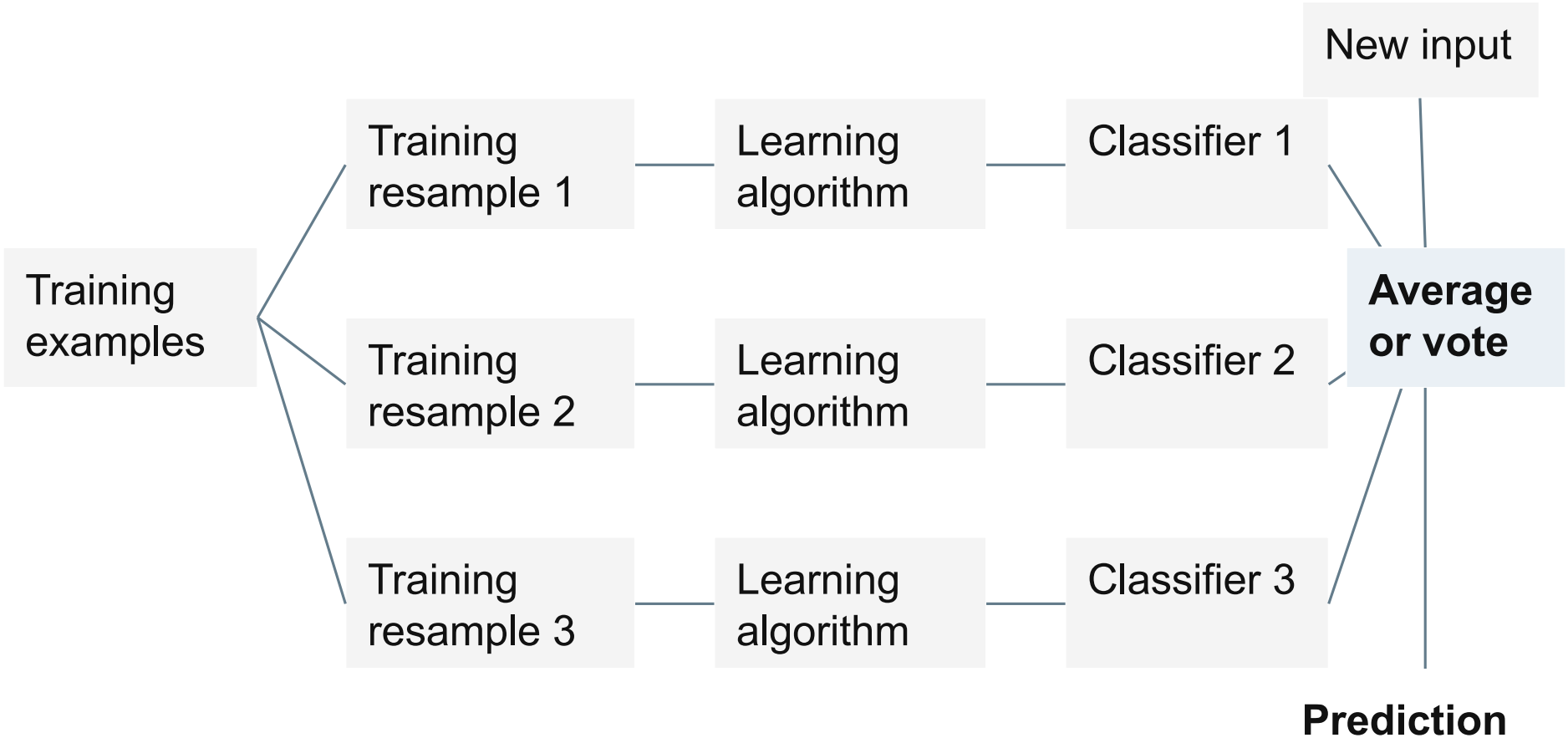
$$E_{\mathrm{COM}} = \frac{1}{K^2}\left(\sum_k \mathbb{E}_{\mathbf{x}}[\epsilon_k^2] + \sum_k \sum_{k' \neq k} \mathbb{E}_{\mathbf{x}}[\epsilon_k \epsilon_{k'}]\right)$$

$$= \frac{1}{K^2}\sum_k E_k = \frac{1}{K} E_{\mathrm{AV}}$$

Averaging reduces independent variation. Shared bias remains.

Bagging uses bootstrap samples

Draw N training examples with replacement for each resample.
Fit one base learner per resample, then combine votes or probabilities.



A bootstrap sample

Six training examples:

1 2 3 4 5 6

Six draws with replacement:

1 1 2 4 4 6 1 and 4 repeat. 3 and 5 are omitted.

Out-of-bag examples: 3 and 5.
Evaluate each using only models that omitted it during fitting.

Resamples share observations, so their model errors can be correlated.

Random forests also vary the candidate features to reduce correlation.

Lecture 2: Classification & optimisation

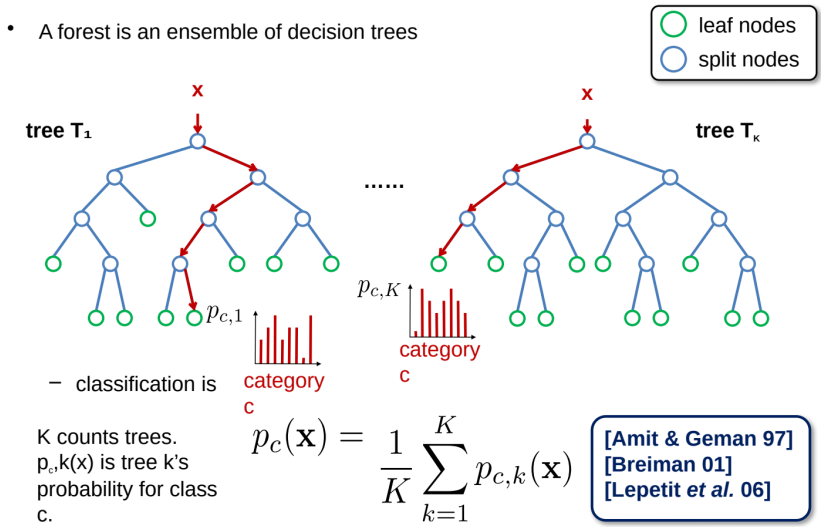
Classification

Optimisation

Trees and ensembles

- Decision trees
- Ensembles and bagging
- **Random forests**

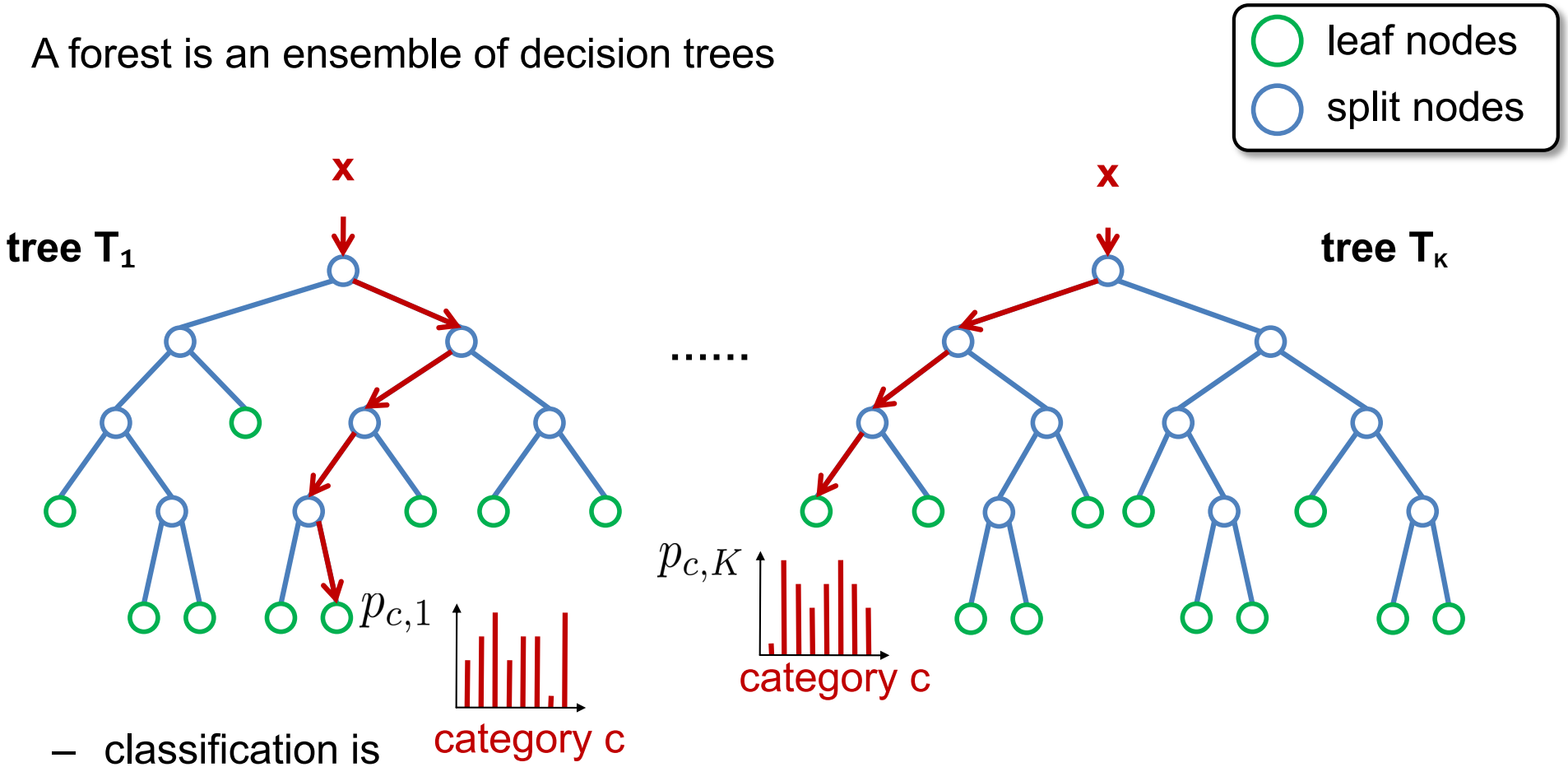
Decisions and evaluation



Murphy, Probabilistic Machine Learning: An Introduction
§18.4: random forests

A forest combines trees

- A forest is an ensemble of decision trees



  - classification is

$$p_c(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} p_{c,k}(\mathbf{x})$$

Training a random forest

- Draw a bootstrap sample to train each tree
- Train each tree:
  - At each node, search a random subset of measured features.
  - Choose the best split within that subset, then recurse.

- Feature variation reduces correlation between tree predictions.
- Combine probabilities or votes.
- Choose complexity using validation or out-of-bag predictions.

Randomising candidate features

Choose a random subset of candidate tests from feature pool F.
For each candidate h, search thresholds in its observed feature range.

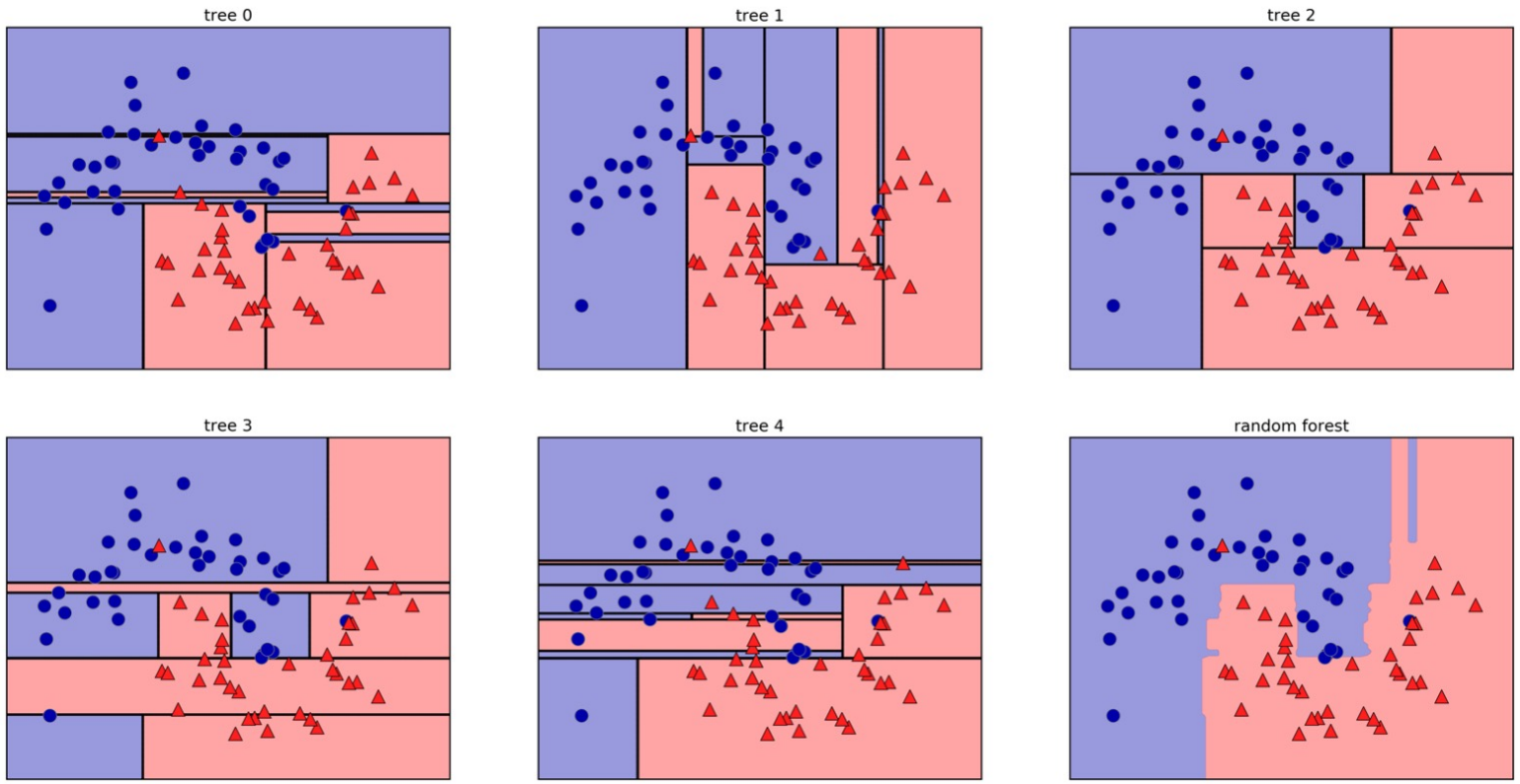
$$\theta \in \left(\min_{i \in I_n} h(\mathbf{x}_i),\ \max_{i \in I_n} h(\mathbf{x}_i) \right]$$

$$I_l = \{ i \in I_n : h(\mathbf{x}_i) < \theta \}$$

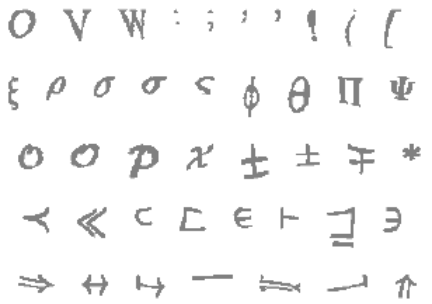
$$I_r = I_n \setminus I_l$$

Choose h and θ to maximise the quality of the child nodes.
For entropy, use the information-gain criterion introduced earlier.
Random feature subsets make the trees less correlated.

A forest composed of 5 trees



Randomized Forests in Vision



[Amit & Geman, 97]
digit recognition



[Lepetit *et al.*, 06]
keypoint recognition



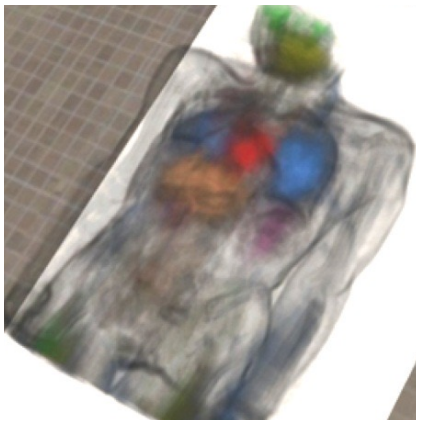
[Moosmann *et al.*, 06]
visual word clustering



[Shotton *et al.*, 08]
object segmentation



[Rogez *et al.*, 08]
pose estimation



[Criminisi *et al.*, 09]
organ detection

Selected applications

Kinect's pipeline







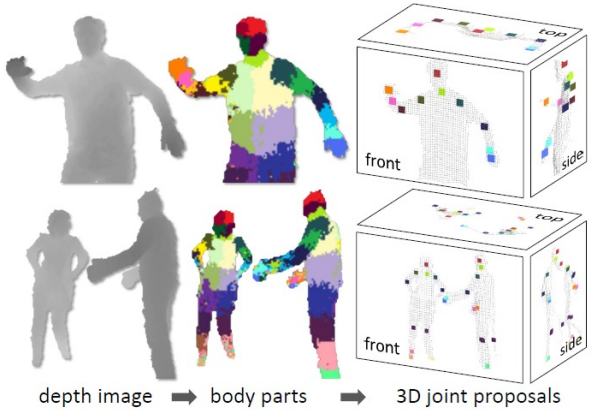


Figure 1. **Overview.** From an single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

A single depth image supplies a classification problem at each pixel.

A random forest predicts body-part labels.

The body-part distributions support 3D joint estimates.

Shotton et al., CVPR 2011: Real-Time Human Pose Recognition in Parts from a Single Depth Image

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

Evaluation metrics

Thresholds and reliability

Decision costs

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.2, 5.1.4: classification and precision–recall

Confusion matrix

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Class 1 is positive. Rows are actual classes and columns are predictions.
TP: true positive. FN: false negative. FP: false positive. TN: true negative.

Total: 8 examples. The matrix shows which kinds of errors occur.

Accuracy

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Accuracy counts all correct predictions.
Here, 5 of the 8 held-out examples have the correct label.

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN} = \frac{5}{8} = 62.5\%$$

Precision

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Among predicted positives, what fraction are truly positive?
False positives reduce precision.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{5} = 60\%$$

Recall

Held-out counts	Predicted 1	Predicted 0
Actual 1	TP = 3	FN = 1
Actual 0	FP = 2	TN = 2

Among actual positives, what fraction did we find?
False negatives reduce recall.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{4} = 75\%$$

Precision and recall

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{5} = 60\%$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{4} = 75\%$$

Pattern	Interpretation
High recall, low precision	Finds most positives, with many false alarms
High precision, low recall	Few false alarms, but misses many positives

F-score

Prec is precision and Rec is recall. F_1 is their harmonic mean.

$$F_1 = \frac{2\,\mathrm{Prec}\,\mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}} = \frac{2TP}{2TP + FP + FN}$$

$$F_b = (1 + b^2)\frac{\mathrm{Prec}\,\mathrm{Rec}}{b^2\mathrm{Prec} + \mathrm{Rec}}, \qquad b > 0$$

b controls the relative importance of recall and precision.
b > 1 gives recall more weight. b < 1 gives precision more weight.
This b is separate from the momentum coefficient β.

Evaluation for both classes

Actual / predicted	Relevant	Not relevant
Relevant	3	5
Not relevant	1	5

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Positive class	Precision	Recall	F_1
Relevant	0.750	0.375	0.500
Not relevant	0.500	0.833	0.625

Accuracy = 8/14 = 0.571. The system misses 5 of 8 relevant articles.

Evaluation with imbalanced classes

Actual / predicted	Relevant	Not relevant
Relevant	0	10
Not relevant	419	571

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Positive class	Precision	Recall	F_1
Relevant	0.000	0.000	0.000
Not relevant	0.983	0.577	0.727

Accuracy = 571/1000 = 0.571, almost unchanged.
All 10 relevant articles are missed (1% of the data).

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

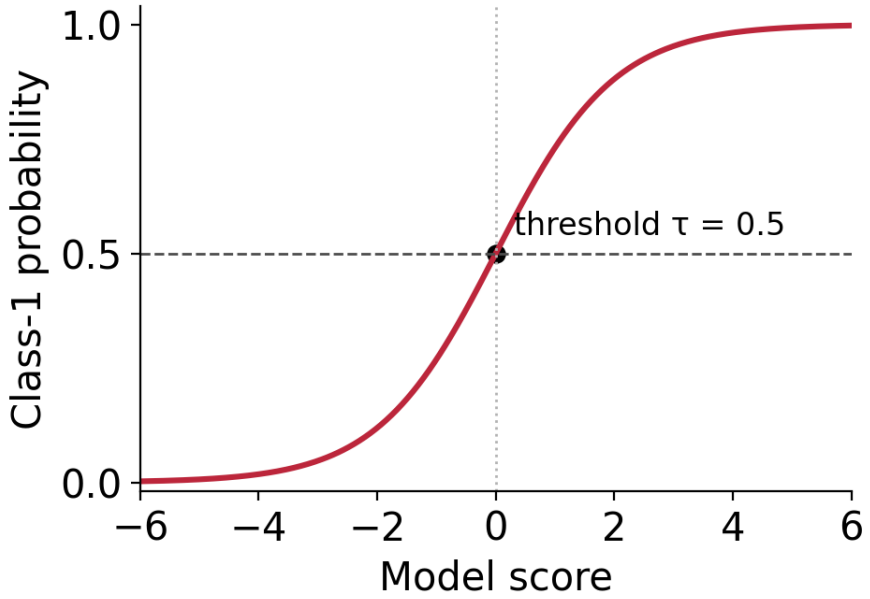
- Evaluation metrics
- **Thresholds and reliability**
- Decision costs

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.3–5.1.4: ROC and precision–recall curves

Classification thresholds

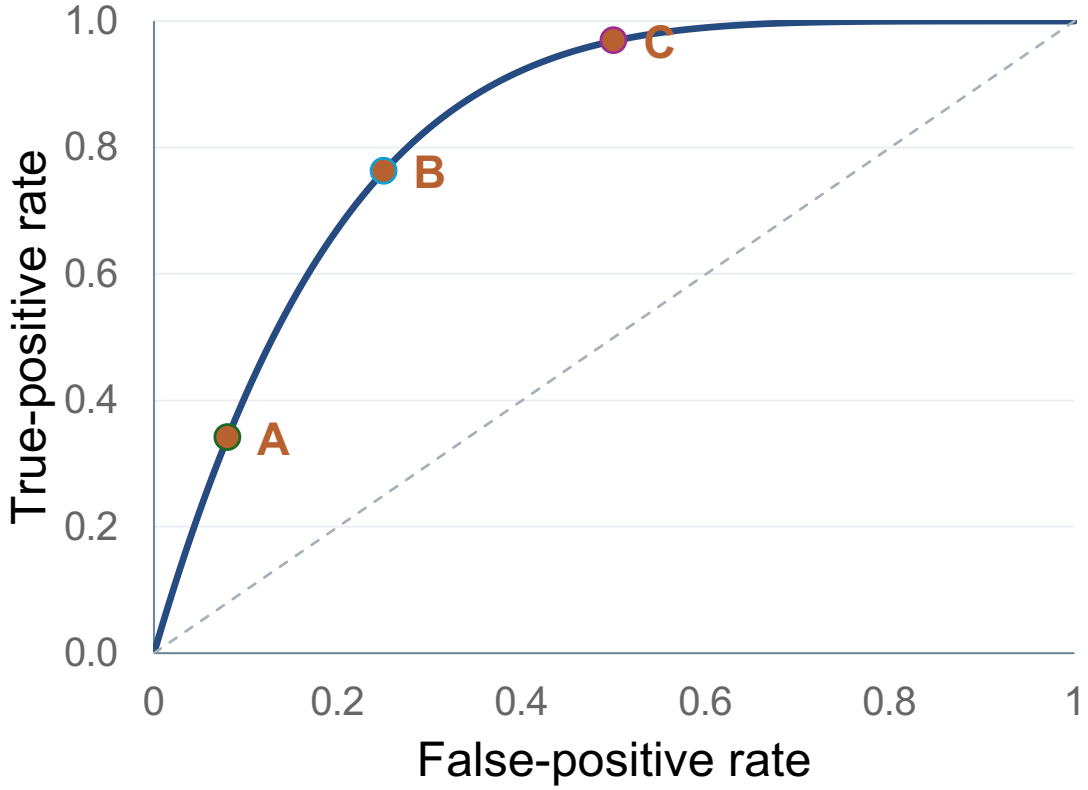
After fitting probabilities, choose how they become decisions.

$$\widehat{y} = \begin{cases} 1 & p_{\mathbf{w}}(\mathbf{x}) \geq \tau \\ 0 & p_{\mathbf{w}}(\mathbf{x}) < \tau \end{cases}$$



τ is the probability threshold. Equal error costs suggest τ = 1/2.
Choose other thresholds using validation data and the cost of mistakes.

ROC curves and operating points



A Higher threshold

Fewer false positives, more missed positives

B Intermediate threshold

More positives detected, more false positives

C Lower threshold

High recall, with more false positives

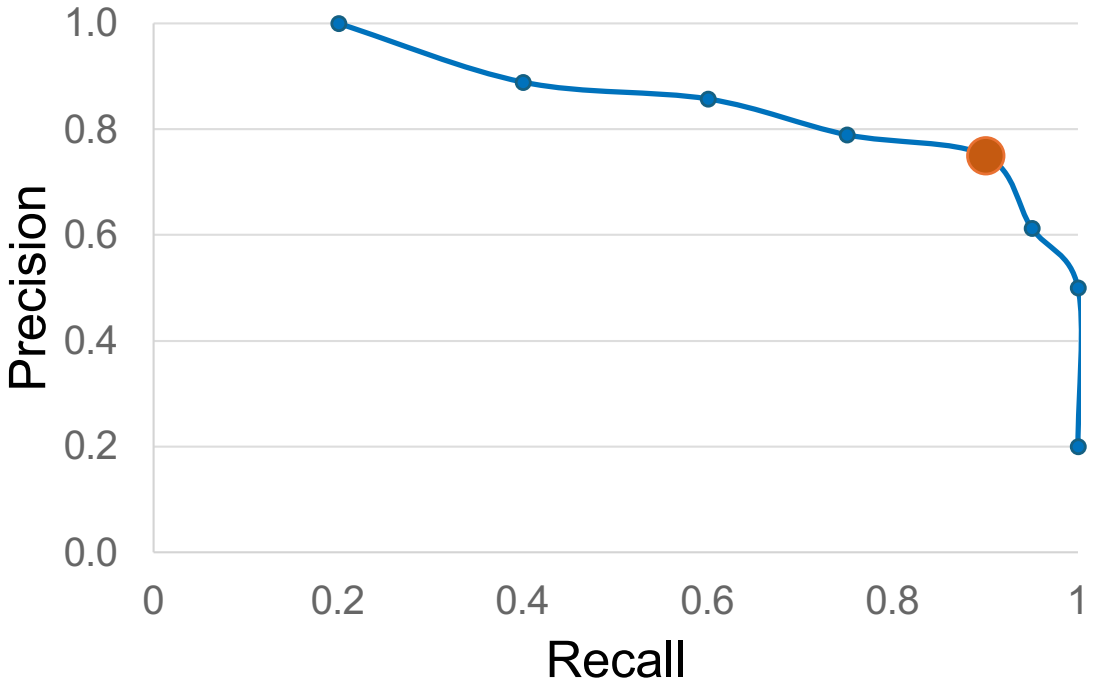
Illustrative ROC curve. Dashed diagonal: chance ranking.

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}}$$

Lowering the threshold moves from A towards C. Choose an operating point using validation data, error costs and class prevalence.

Precision–recall and the F_1 optimum

Sweep the threshold on validation data. Each point is one threshold.



Maximum F_1

Threshold τ = 0.50

Precision = 0.75
Recall = 0.90
$F_1 \approx 0.818$

$$F_1 = \frac{2PR}{P + R}$$

Illustration: 20 positives and 80 negatives

Set the threshold at the highest validation F_1 for the best tradeoff when precision and recall carry equal weight.

Freeze this threshold, then evaluate on independent test data.

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

- Evaluation metrics
- Thresholds and reliability
- **Decision costs**

Truth / decision	Class 0	Class 1
Class 0	Λ_{00}	Λ_{01}
Class 1	Λ_{10}	Λ_{11}

Murphy, Probabilistic Machine Learning: An Introduction
§§5.1.1–5.1.2: decision theory and classification

Classes and decisions

The true class describes the input. A decision specifies an action.

Medical example	Possible values
True class	Sick or healthy
Decision	Treat, discharge, or request another examination

An extra examination is an action even though it is not a third true class.
Use a loss function to express the consequence of each action.

Asymmetric decision costs

A false negative and a false positive can have very different consequences.

True condition	Decision	Consequence
Sick	Healthy	Missed illness
Healthy	Sick	Unnecessary treatment

In this example, missing an illness carries a larger loss.
The preferred decision depends on both probabilities and error costs.

A loss matrix for decisions

$$\Lambda_{kj} = \text{loss for decision } j \text{ when the true class is } k$$

Truth / decision	Class 0	Class 1
Class 0	Λ_{00}	Λ_{01}
Class 1	Λ_{10}	Λ_{11}

k indexes the true class and j the decision. Rows describe truth. Columns describe actions. R is the expected loss of an action.

$$R(j \mid \mathbf{x}) = \sum_{k=0}^{1} \Lambda_{kj}\, p_k(\mathbf{x}) \qquad \widehat{j} = \underset{j}{\operatorname{argmin}}\ R(j \mid \mathbf{x})$$

Decision costs: the diagnosis example

An illustrative loss matrix from the course example

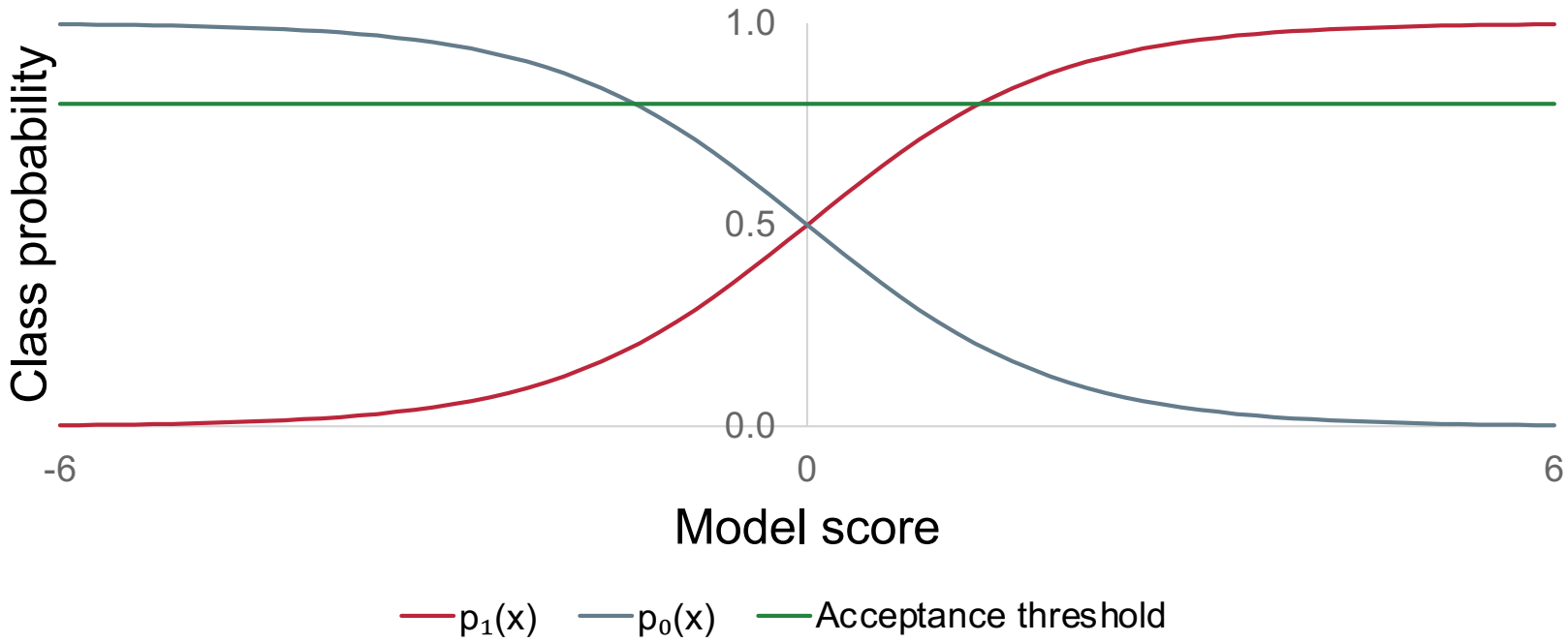
Truth / decision	Cancer (1)	Normal (0)
Cancer (1)	0	1000
Normal (0)	1	0

$$p = p_1(\mathbf{x}) = P(Y = 1 \mid \mathbf{x})$$

$$R(1 \mid \mathbf{x}) = 1 - p, \quad R(0 \mid \mathbf{x}) = 1000p$$

$$\text{Choose class 1 when } p \geq \frac{1}{1001} \approx 0.001$$

A reject option



$$\text{Accept a class if } \max\{p_0(\mathbf{x}), p_1(\mathbf{x})\} \geq \kappa$$

Two classes, three actions: class 0, class 1, or reject.
Choose the most probable class if its probability is at least κ, with $1/2 < \kappa \leq 1$.
Otherwise reject, for further testing or a human decision.

Classification: fit, decide, evaluate

Fit a score or class distribution

Choose a loss that suits classification. Optimization or tree splitting fits the model to the training data.

Choose an action

Class probabilities and error costs determine the decision. A threshold of 1/2 assumes equal costs in the binary case.

Assess unseen performance

Use validation for choices, then independent test data for assessment. Match the metric to the task and check probability calibration.

Lecture 2: Classification & optimisation

Classification

Optimisation

Trees and ensembles

Decisions and evaluation

Appendix

Further derivations and Lecture 1 reference material

Signed-label probabilities

Newton–Raphson for logistic regression

Summation and matrix expressions

Iteratively reweighted least squares (IRLS)

Multiclass objective

Murphy, Probabilistic Machine Learning: An Introduction
§10.2.6: iteratively reweighted least squares

Newton for logistic regression gives IRLS

At iteration t, evaluate probabilities and curvature weights:

$$\mathbf{p} = g(\mathbf{X}\mathbf{w}_t), \qquad \mathbf{R} = \mathrm{diag}[p_i(1-p_i)]$$

Substitute $\nabla L = X^T(p - y)$ and $H = X^TRX$ into the Newton step:

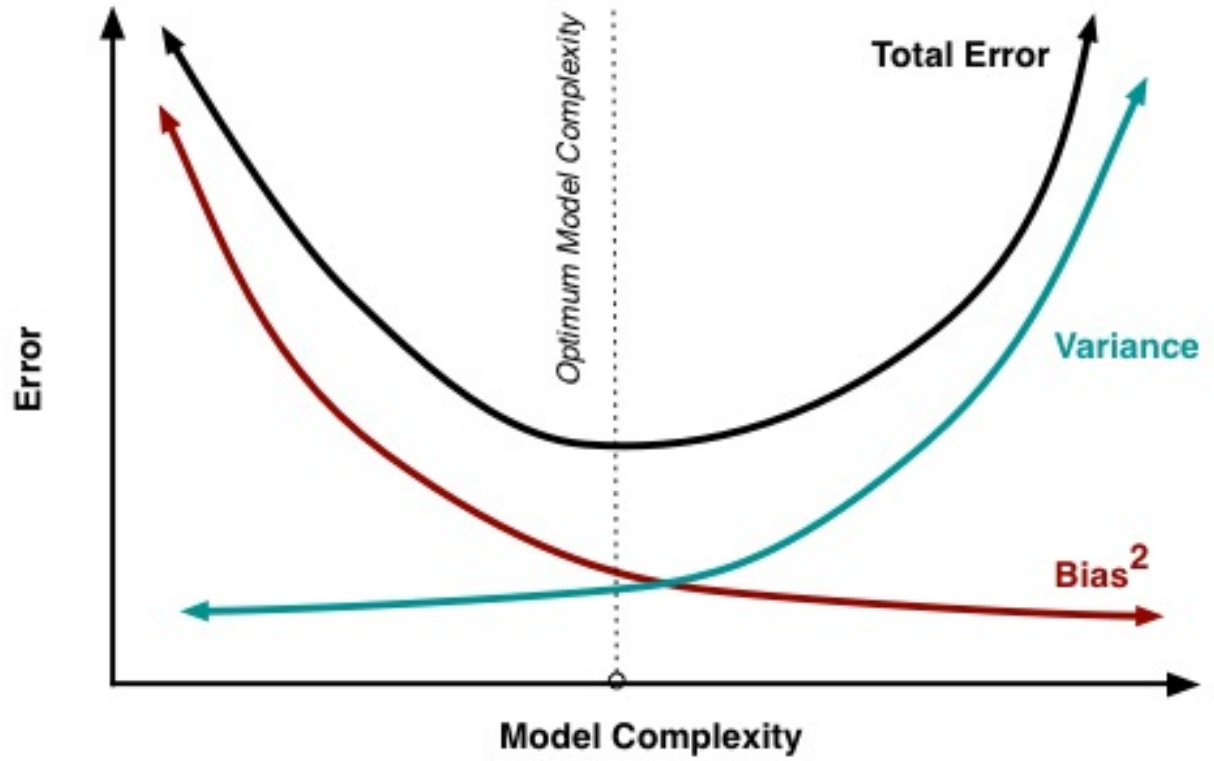
$$(\mathbf{X}^T\mathbf{R}\mathbf{X})\mathbf{w}_{t+1} = \mathbf{X}^T\mathbf{R}\mathbf{X}\mathbf{w}_t + \mathbf{X}^T(\mathbf{y} - \mathbf{p})$$

Define a working response u. The same update solves weighted least squares:

$$\mathbf{u} = \mathbf{X}\mathbf{w}_t + \mathbf{R}^{-1}(\mathbf{y} - \mathbf{p})$$

$$\mathbf{w}_{t+1} = \arg\min_{\mathbf{w}} \frac{1}{2}(\mathbf{u} - \mathbf{X}\mathbf{w})^T\mathbf{R}(\mathbf{u} - \mathbf{X}\mathbf{w})$$

Bias-variance decomposition



Empirical risk vs population risk

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \qquad \ell(y, \hat{y}) = (y - \hat{y})^2$$

Training risk: mean squared error on the training set:

$$\hat{R}_{\text{train}}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, f(\mathbf{x}_i))$$

Empirical risk minimization:

$$\hat{f}_{\mathcal{D}} \in \arg\min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$$

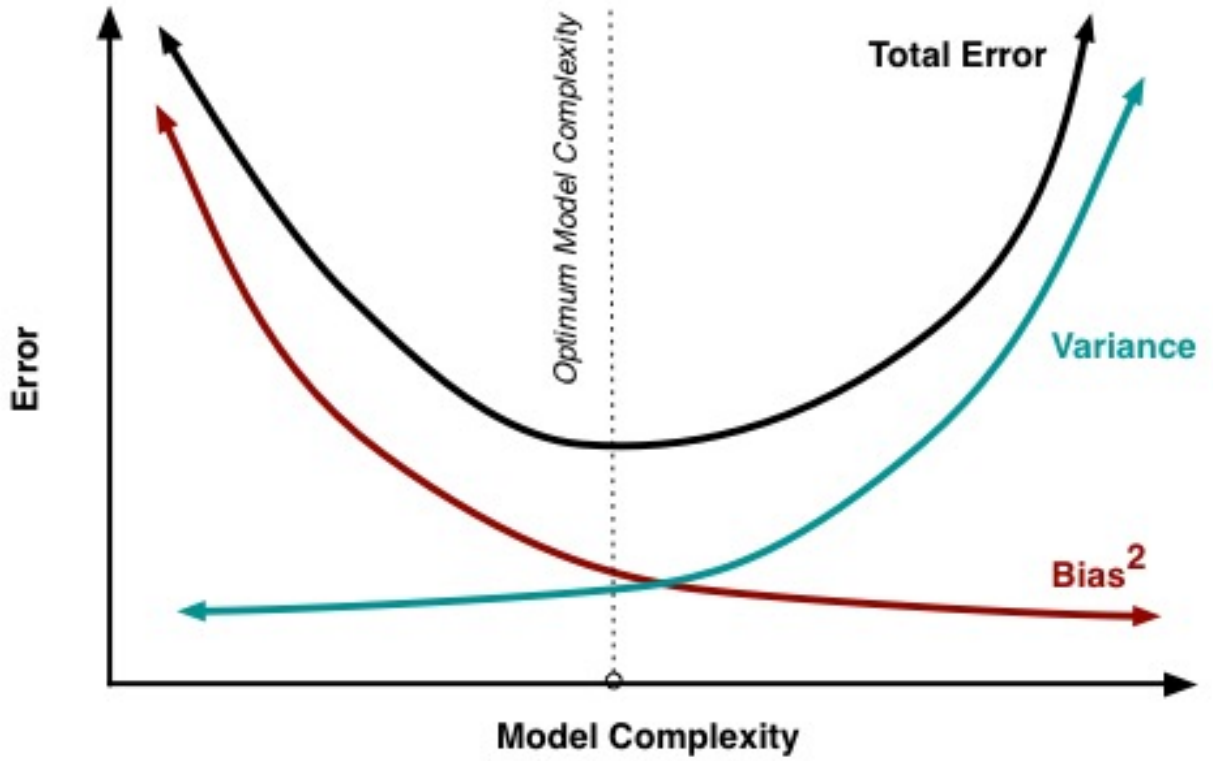
Population risk:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P}[\ell(Y, f(\mathbf{X}))] = \iint \ell(y, f(\mathbf{x}))\, p(\mathbf{x}, y)\, \mathrm{d}y\, \mathrm{d}\mathbf{x}$$

F: candidate function class.
Subscript D: the training set used for this fit.
p(x, y): joint density for continuous inputs and targets.

Bias-variance decomposition

$$\mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})] = \mathbb{E}_{\mathbf{X}}[\text{bias}^2(\mathbf{X}) + \text{variance}(\mathbf{X}) + \text{noise}(\mathbf{X})]$$



Bias–variance decomposition at one input

$$Y = f^*(x) + \epsilon, \quad \mathbb{E}[\epsilon \mid x] = 0, \quad \mathbb{E}[\epsilon^2 \mid x] = \sigma^2(x)$$

Add and subtract the true mean and the mean fit

$$Y - \hat{f}_{\mathcal{D}}(x) = \underbrace{f^*(x) - m(x)}_{\text{mean error}} + \underbrace{m(x) - \hat{f}_{\mathcal{D}}(x)}_{\text{fit fluctuation}} + \epsilon$$

Cross-terms vanish: the fluctuations have mean zero, noise is independent of the fitted predictor. $\mathbb{E}_{\mathcal{D}}[m(x) - \hat{f}_{\mathcal{D}}(x)] = 0, \qquad \mathbb{E}[\epsilon \mid x] = 0$

$$\mathbb{E}_{\mathcal{D},Y}[(Y - \hat{f}_{\mathcal{D}}(x))^2 \mid X = x]$$

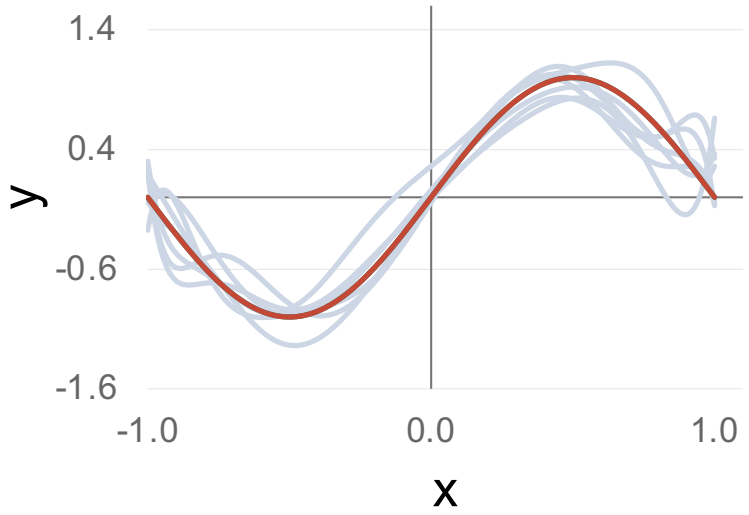
$$= \underbrace{(m(x) - f^*(x))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - m(x))^2]}_{\text{variance}} + \underbrace{\sigma^2(x)}_{\text{noise}}$$

Target noise ε is independent of the training data. σ²(x) is noise variance at x.

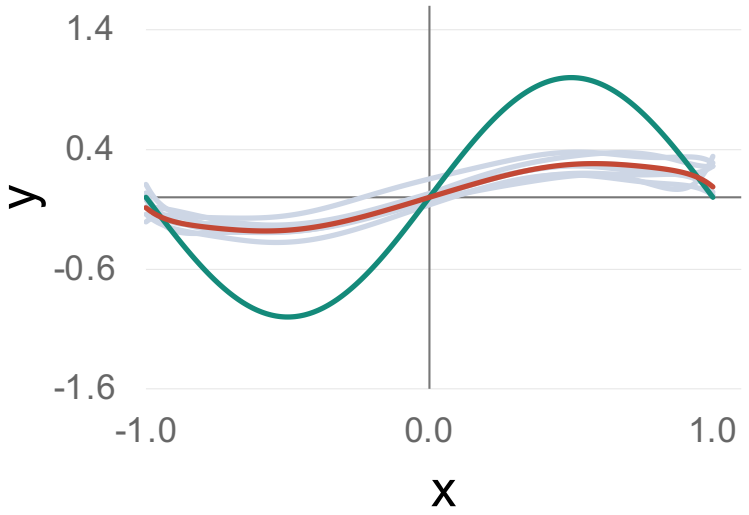
What does “averaging over datasets” mean?

Repeat collecting N examples from the same source and refitting.
Keep the function class and ridge strength λ fixed within each plot.

Weak ridge: λ = 0.0001



Strong ridge: λ = 10



At scalar input x: grey fits, red mean fit m(x), green true mean f*(x).

$$m(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)], \qquad f^*(x) = \mathbb{E}[Y \mid X = x]$$

E with subscript D averages training sets.
Here: fixed inputs, fresh noisy targets. With random inputs, resample those too.

Splitting the examples at a node

At node n, I_n indexes labelled training examples (x_i, y_i).
$P_n(c)$ is the histogram of their labels: the fraction belonging to class c.

$$P_n(c) = \frac{1}{|I_n|} \sum_{i \in I_n} \mathbb{I}[y_i = c]$$

$$I_l = \{i \in I_n : h(\mathbf{x}_i) < \theta\}$$

$$I_r = I_n \setminus I_l$$

$h(x_i)$ tests a feature of example i. θ is its threshold.
Each example follows one branch. Recursively split the child sets.

Reading alongside Lecture 2

Murphy, Probabilistic Machine Learning: An Introduction (book1.pdf)

Classification

§§10.2.1–10.2.3, 10.3.1–10.3.2

Vision Book: §§9.6, 9.7.3 — classification and cross-entropy

Optimisation

§§8.2.1–8.2.4, 8.3.1, 8.4.1–8.4.3; §§11.4.1–11.4.2 (L1)

Vision Book: §§10.3–10.5, 10.7 — gradient descent, momentum and SGD

Trees and ensembles

§4.7.6; §§18.1, 18.3–18.4

Decisions and evaluation

§§5.1.1–5.1.4

Vision Book: Foundations of Computer Vision — Torralba, Isola and Freeman
visionbook.mit.edu

Notation: models and optimisation

Data and model

$N,\ D,\ M$

Examples; raw/mapped size

$\mathbf{x}_i,\ y_i$

Example i: input and observed label

$\boldsymbol{\phi}(\mathbf{x}),\ \mathbf{w}$

Features and coefficients

$z_i,\ g$

Score and logistic sigmoid

$p_i,\ p_c(\mathbf{x})$

Class-1 and class-c probabilities

$y_{\pm},\ m = y_{\pm} z$

Signed label (±1) and margin

Fitting and optimisation

$\ell_i,\ L,\ J$

Example loss; total loss; objective

$\lambda,\ \mathbf{r}(\mathbf{w})$

Penalty strength; penalty gradient

$\nabla_{\mathbf{w}} J,\ \mathbf{H}$

Gradient and Hessian

$t,\ \eta_t,\ \alpha_t$

Iteration; learning rate; step scale

$\mathcal{B}_t,\ B$

Mini-batch indices and size

$\mathbf{g}_t,\ \mathbf{v}_t,\ \beta$

Gradient estimate; momentum; decay

Notation: trees and decisions

Trees and ensembles

$C,\ K$

Classes; learners

$f_k,\ \bar{f}$

Base learner; averaged prediction

$\mathcal{R},\ p_c(\mathbf{x})$

Leaf region and class probability

$\theta_n,\ H,\ \mathrm{IG}$

Split threshold; entropy; split gain

$f^*(x),\ m(x)$

True mean; mean over datasets

$R(f),\ \widehat{R}_{\mathrm{train}}$

Population risk; training risk

Decisions and evaluation

$\tau,\ \kappa$

Decision and acceptance thresholds

$\Lambda_{kj},\ R(j \mid \mathbf{x})$

Cost (truth k, action j); risk

$\mathrm{TP},\ \mathrm{FP},\ \mathrm{TN},\ \mathrm{FN}$

True/false positives and negatives

$\mathrm{Prec},\ \mathrm{Rec},\ F_b$

Precision, recall, weighted F-score

$\mathrm{TPR},\ \mathrm{FPR},\ \mathrm{AUC}$

Positive rates; area under ROC

$e,\ N_{\mathrm{test}}$

Test error fraction; test-set size